

Wymagający złamania wierszy tytuł pracy w języku polskim

(English title)

Maksymilian Debeściak

Praca licencjacka

Promotor: dr Jan Kowalski

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

5 maja 2026

Streszczenie

...



...

Spis treści

1. Wprowadzenie	9
1.1. Dedukcja naturalna	9
1.2. Zapis tekstowy dowodów	11
2. Reprezentacja formuł, sekwentów, reguł wnioskowania i dowodów	17
2.1. Reprezentacja formuł i sekwentów	17
2.2. Reprezentacja reguł wnioskowania	18
2.3. Reprezentacja dowodów	19
3. Strategie konstruowania dowodów	23
3.1. Motywacja	23
3.2. Strategia pierwsza: liniowe dopisywanie kroków dowodu	24
3.2.1. Idea strategii	24
3.2.2. Generowanie i weryfikacja pojedynczej linii	25
3.2.3. Warunki zakończenia	27
3.2.4. Ograniczenia strategii	27
3.3. Strategia druga: przeszukiwanie wielu częściowych dowodów	28
3.3.1. Motywacja	28
3.3.2. Zbiór częściowych dowodów	28
3.3.3. Ocena prawdopodobieństwa częściowego dowodu	29
3.3.4. Pełny przebieg strategii	30
3.3.5. Ograniczenia strategii	31
3.4. Strategia trzecia: dekompozycja celu i drzewo poszukiwania	32

3.4.1.	Motywacja	32
3.4.2.	Rozbicie problemu na podproblemy	33
3.4.3.	Format zapytania o rozbicie	34
3.4.4.	Sprawdzanie poprawności rozbicia	35
3.4.5.	Generowanie rozbić przez model	36
3.4.6.	Dwa typy wierzchołków drzewa	36
3.4.7.	Wartość wierzchołka	37
3.4.8.	Rozwijanie liścia	38
3.4.9.	Rekurencyjne przeszukiwanie drzewa	38
3.4.10.	Sklejanie dowodów	39
3.5.	Strategia trzecia: dekompozycja celu i drzewo poszukiwania	42
3.5.1.	Motywacja	42
3.5.2.	Rozbicie problemu na podproblemy	42
3.5.3.	Format zapytania o rozbicie	42
3.5.4.	Sprawdzanie poprawności rozbicia	42
3.5.5.	Generowanie rozbić przez model	42
3.5.6.	Dwa typy wierzchołków drzewa	42
3.5.7.	Wartość wierzchołka	42
3.5.8.	Rozwijanie liścia	42
3.5.9.	Rekurencyjne przeszukiwanie drzewa	42
3.6.	Podsumowanie strategii	42
4.	3 strategie	43
5.	Rozbijanie problemów na podproblemy	47
5.1.	Obserwacja	47
6.	Klasa node	51
7.	Generowanie korpusu	55
8.	Trenowanie i korzystanie z modelu (NanoGPTBundle i te wszystkie funkcje z nim związane !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!, podkreślam WSZYST-	

KIE(liczenie prawdopodobieństwa, dodawanie tokenu, linii, trenowanie, dotrenowanie, ładowanie)!!!!)	57
--	-----------

Rozdział 1.

Wprowadzenie

Dedukcja naturalna jest metodą dowodzenia twierdzeń logicznych przez wychodzenie z pewnych założeń i modyfikowanie ich za pomocą ściśle zdefiniowanych reguł wnioskowania.

1.1. Dedukcja naturalna

Definicja 1. Sekwentem nazywamy wyrażenie postaci

$$\Gamma \vdash \varphi$$

gdzie Γ jest skończonym zbiorem formuł (zapisywanych po przecinku i bez nawiasów klamrowych) zwanym *kontekstem*, a φ jest formułą zwaną *tezą sekwentu*. Sekwent $\Gamma \vdash \varphi$ interpretujemy jako stwierdzenie, że jeśli założymy, że wszystkie formuły z Γ są prawdziwe, da się dowieść φ . W rachunku zdań w logice klasycznej, który będzie jedyną logiką w której się będziemy poruszać, zdanie prawdziwe we wszystkich wartościowaniach jest dowodliwe, tak więc $\Gamma \vdash \phi$ jest równoważne stwierdzeniu jeżeli wszystkie formuły kontekstu Γ są prawdziwe to ϕ jest prawdziwe.

Definicja 2. Regułą wnioskowania nazywamy schemat postaci

$$\frac{S_1 \quad S_2 \quad \dots \quad S_n}{S}(R)$$

gdzie $S_1, \dots, S_n, S$ są sekwentami. Sekwenty $S_1, \dots, S_n$ nazywamy *przesłankami reguły*, natomiast sekwent S nazywamy *konkluzją reguły*. R to z kolei *indeks reguły*, który służy do jej identyfikacji. Jeżeli przesłanki reguły są prawdziwe to z definicji jej konkluzja jest również prawdziwa.

Dowody w dedukcji naturalnej można utożsamiać z drzewami, których węzłami są sekwenty, a krawędzie są indeksowane indeksami reguł wnioskowania.

W nieniejszej pracy skupimy się na dowodzeniu twierdzeń w logice klasycznej z następującymi spójnikami logicznymi: koniunkcją ($\wedge$), alternatywą ($\vee$), negacją ($\neg$) oraz implikacją ($\rightarrow$). regułami wnioskowania:

$\frac{}{\varphi \vdash \varphi} (\text{Ass})$	$\frac{\Gamma \vdash \varphi}{\Gamma, \psi \vdash \varphi} (\text{W})$
$\frac{\Gamma, \varphi \vdash \psi}{\Gamma \vdash \varphi \rightarrow \psi} (\rightarrow I)$	$\frac{\Gamma_1 \vdash \varphi \rightarrow \psi \quad \Gamma_2 \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \psi} (\rightarrow E)$
$\frac{\Gamma, \varphi \vdash \perp}{\Gamma \vdash \neg \varphi} (\neg I)$	$\frac{\Gamma_1 \vdash \neg \varphi \quad \Gamma_2 \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \perp} (\neg E)$
$\frac{\Gamma_1 \vdash \varphi \quad \Gamma_2 \vdash \psi}{\Gamma_1, \Gamma_2 \vdash \varphi \wedge \psi} (\wedge I)$	$\frac{\Gamma \vdash \varphi \wedge \psi}{\Gamma \vdash \varphi} (\wedge E_1)$
$\frac{\Gamma \vdash \varphi}{\Gamma \vdash \varphi \vee \psi} (\vee I_1)$	$\frac{\Gamma \vdash \varphi \wedge \psi}{\Gamma \vdash \psi} (\wedge E_2)$
$\frac{\Gamma \vdash \psi}{\Gamma \vdash \varphi \vee \psi} (\vee I_2)$	$\frac{\Gamma_1 \vdash \varphi \vee \psi \quad \Gamma_2, \varphi \vdash \rho \quad \Gamma_3, \psi \vdash \rho}{\Gamma_1, \Gamma_2, \Gamma_3 \vdash \rho} (\vee E)$
$\frac{}{\vdash \top} (\top I)$	$\frac{\Gamma \vdash \perp}{\Gamma \vdash \varphi} (\perp E)$
$\frac{\Gamma_1, \varphi \vdash \psi \quad \Gamma_2, \psi \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \varphi \leftrightarrow \psi} (\leftrightarrow I)$	$\frac{\Gamma_1 \vdash \varphi \leftrightarrow \psi \quad \Gamma_2 \vdash \varphi}{\Gamma_1, \Gamma_2 \vdash \psi} (\leftrightarrow E_1)$
$\frac{\Gamma_1 \vdash \varphi \leftrightarrow \psi \quad \Gamma_2 \vdash \psi}{\Gamma_1, \Gamma_2 \vdash \varphi} (\leftrightarrow E_2)$	$\frac{\Gamma, \neg \varphi \vdash \perp}{\Gamma \vdash \varphi} (\text{RAA})$
$\frac{\Gamma \vdash \neg \neg \varphi}{\Gamma \vdash \varphi} (\neg \neg)$	$\frac{}{\vdash \varphi \vee \neg \varphi} (\text{TND})$

Tabela 1.1: Reguły wnioskowania rozważanego systemu

Oto przykładowy dowód w dedukcji naturalnej w notacji drzewiastej:

$$\frac{\frac{\frac{}{a \vee b \vdash a \vee b} (\text{Ass}) \quad \frac{\frac{}{a \vdash a} (\text{Ass})}{a \vdash b \vee a} (\vee I_2)}{a \vee b \vdash b \vee a} (\vee E) \quad \frac{}{b \vdash b} (\text{Ass})}{b \vdash b \vee a} (\vee I_1)}{\vdash (a \vee b) \rightarrow (b \vee a)} (\rightarrow I)$$

Dużą zaletą dedukcji naturalnej jest to, że niejako symuluje ona sposób, w jaki ludzie przeprowadzają wnioskowanie, zarówno w matematyce, jak i w codziennym życiu. Dowody mają swoją drzewiastą strukturę, którą można łatwo analizować. Inną zaletą tej metody jest jej uniwersalność. Dodając odpowiednie reguły wnioskowania możemy otrzymać dedukcję naturalną dla różnych systemów logicznych np logiki pierwszego i drugiego rzędu, czy logiki modalnej. Jeszcze jedną zaletą jest to, że w przypadku większości użytecznych dowodów jesteśmy w stanie uniknąć wykładniczo dużej złożoności wielu innych systemów dowodzenia sprowadzających się

do rozpatrzenia wszystkich możliwych przypadków. Z drugiej strony dedukcja naturalna jest trudna w zaimplementowaniu. Zwykle dowód przeprowadzamy z góry na dół, zaczynając od założeń i kończąc na tezie. Jednak nie zawsze jasne jest, którą regułę wnioskowania zastosować i co gorsza wyniki zastosowania niektórych reguł mogą zawierać podformuły których nie ma w przesłankach, lub jej wybór nie jest jednoznaczny. Np. używając reguły

$$\frac{\Gamma \vdash \varphi}{\Gamma \vdash \varphi \vee \psi} (\vee I_1)$$

w przesłance nie mamy informacji o formule ψ , która pojawi się w konkluzji. Jednak podobieństwo dedukcji naturalnej do ludzkiego sposobu wnioskowania sprawia, oraz pewien dający się zauważyć porządek w zapisie dowodów sprawia, że gdzie klasyczna implementacja dedukcji naturalnej jest trudna, wydaje się ona dobrze nadawać do implementacji z użyciem metod sztucznej inteligencji, w tym modeli językowych o architekturze transformerów.

W niniejszej pracy przedstawimy implementację automatycznego systemu dowodzenia twierdzeń w logice klasycznej przy użyciu odpowiednio wytrenowanego modelu nano-GPT oraz przedstawimy wyniki jego działania i jak dobrze radzi sobie z przeprowadzaniem dowodów. Konstrukcja systemu będzie opisana krok po kroku, zaczynając od zaprogramowania logicznej struktury systemu, tłumaczenia zapisu dowodu w formie tekstowej na strukturę logiczną i sprawdzanie poprawności dowodu, generacja korpusu treningowego, trenowanie modelu oraz kolejne etapy konstrukcji samego systemu dowodzenia (wybór możliwych linii, równoległość prowadzenia kilku ścieżek rozumowania, itp.).

1.2. Zapis tekstowy dowodów

Notacja Fitcha jest sposobem zapisu dowodów dedukcji naturalnej w formie tekstowej. Jest on wygodny, ponieważ metoda zapisywania drzew dowodu potrafi zajmować bardzo dużo miejsca i trudno użyć do niej modeli językowych.

Z punktu widzenia implementacji komputerowej klasyczna notacja Fitcha narzuca ograniczenia wynikające ze struktury zagnieżdżonych bloków, które określają, do których założeń można się odwoływać w danym miejscu dowodu. W konsekwencji dostęp do wcześniejszych kroków dowodu zależy od ich położenia w tej strukturze.

W stosowanej w niniejszej pracy notacji dowód koduje uporządkowaną listę sekwentów w, której każda linia koduje jeden sekwent, a zależności między kolejnymi krokami są wyrażane poprzez odwołania do indeksów wcześniejszych linii oraz zastosowane reguły wnioskowania. Nie występują tu ograniczenia wynikające ze zagnieżdżonych bloków, a dowolny wcześniejszy sekwent może zostać użyty jako przesłanka, o ile spełnia warunki danej reguły.

Takie podejście pozwala traktować konstrukcję dowodu jako proces składania sekwentów na podstawie wcześniej uzyskanych wyników, bez konieczności odtwarzania struktury zagnieżdżeń charakterystycznej dla klasycznej notacji Fitcha.

Co więcej, w stosowanej notacji każdej linii zapisywana jest jawnie teza sekwentu, który dana linia koduje. Jest to użyteczne podczas sprawdzania poprawności dowodu.

Podejście to wprowadza jednak pewną niedogodność: nie istnieje jedna reguła, pozwalająca wyciągnąć formułę należącą do kontekstu sekwentu w jednym kroku. Ponieważ jest to dość częsta operacja wprowadzamy dwie pomocnicze reguły, których celem nie jest wprowadzanie nowych własności logicznych systemu, lecz stanowią wygodny mechanizm umożliwiający prostsze manipulowanie kontekstem.

$$\frac{\varphi, \Gamma \vdash \psi}{\Gamma, \varphi \vdash \varphi} \text{ (FromContext)} \quad \frac{\Gamma \vdash \psi}{\Gamma, \varphi \vdash \varphi} \text{ (FromWeakenContext)}$$

Poniżej przedstawiam drzewa dowodu poprawności tych reguł:

$$\frac{\frac{\Gamma, \varphi \vdash \psi \quad \frac{}{\varphi \vdash \varphi} \text{ (Ass)}}{\Gamma, \varphi \vdash \psi \wedge \varphi} \text{ (\wedge I)}}{\Gamma, \varphi \vdash \varphi} \text{ (\wedge E}_2\text{)}$$

Dowód reguły (FromContext)

$$\frac{\frac{\Gamma \vdash \psi \quad \frac{}{\varphi \vdash \varphi} \text{ (Ass)}}{\Gamma, \varphi \vdash \psi \wedge \varphi} \text{ (\wedge I)}}{\Gamma, \varphi \vdash \varphi} \text{ (\wedge E}_2\text{)}$$

Dowód reguły (FromWeakenContext)

Aby sformalizować podejście, w którym każda linia dowodu koduje jeden sekwent, wprowadzamy oznaczenia pozwalające jednoznacznie interpretować poszczególne linie dowodu oraz odwołania między nimi:

`Fitch(i)` := sekwent kodowany przez linię o indeksie `i`

oraz oznaczenie

`s.conclusion` := teza sekwentu `s`

`s.assumptions` := kontekst sekwentu `s`

Poniżej przedstawimy interpretację poszczególnych reguł wnioskowania w notacji Fitcha, a także ich odpowiedniki w notacji drzewiastej:

Notacja Fitcha	Notacja drzewiasta
i. ϕ assumption	$\overline{\phi \vdash \phi} \text{ (Ass)}$
j. ψ k. ϕ i. ϕ weakening, j, k	$\frac{\text{Fitch}(k)}{\text{Fitch}(i)} (W)$ gdzie $\text{Fitch}(i).\text{assumptions} =$ $\text{Fitch}(k).\text{assumptions},$ $\text{Fitch}(j).\text{conclusion}$
j. ϕ k. ψ i. $\phi \rightarrow \psi$ $\rightarrow$ -introduction, j-k	$\frac{\text{Fitch}(k)}{\text{Fitch}(i)} (\rightarrow I)$
j. ϕ k. $\perp$ i. $\neg\phi$ $\neg$ -introduction, j-k	$\frac{\text{Fitch}(k)}{\text{Fitch}(i)} (\neg I)$
j. ϕ k. $\phi \rightarrow \psi$ i. ψ $\rightarrow$ -elimination k, j	$\frac{\text{Fitch}(k) \quad \text{Fitch}(j)}{\text{Fitch}(i)} (\rightarrow E)$
j. $\neg\phi$ k. ϕ i. $\perp$ $\neg$ -elimination, j, k	$\frac{\text{Fitch}(j) \quad \text{Fitch}(k)}{\text{Fitch}(i)} (\neg E)$
j. ϕ k. ψ i. $\phi \wedge \psi$ $\wedge$ -introduction, j, k	$\frac{\text{Fitch}(j) \quad \text{Fitch}(k)}{\text{Fitch}(i)} (\wedge I)$
j. ϕ i. $\phi \vee \psi$ $\vee$ -introduction1, j	$\frac{\text{Fitch}(j)}{\text{Fitch}(i)} (\vee I_1)$

j. $\psi \quad \dots$ $\dots$ i. $\phi \vee \psi \quad \vee\text{-introduction2, } j$	$\frac{\text{Fitch}(j)}{\text{Fitch}(i)} (\vee I_2)$
i. $\top \quad \top\text{-introduction}$	$\overline{\vdash \top} (\top I)$
j. $\phi \wedge \psi \quad \dots$ $\dots$ i. $\phi \quad \wedge\text{-elimination1, } j$	$\frac{\text{Fitch}(j)}{\text{Fitch}(i)} (\wedge E_1)$
j. $\phi \wedge \psi \quad \dots$ $\dots$ i. $\psi \quad \wedge\text{-elimination2, } j$	$\frac{\text{Fitch}(j)}{\text{Fitch}(i)} (\wedge E_2)$
j. $\phi \vee \psi \quad \dots$ $\dots$ k. $\rho \quad \dots$ $\dots$ l. $\rho \quad \dots$ $\dots$ i. $\rho \quad \vee\text{-elimination, } j, k, l$	$\frac{\text{Fitch}(j) \quad \text{Fitch}(k) \quad \text{Fitch}(l)}{\text{Fitch}(i)} (\vee E)$
j. $\perp \quad \dots$ $\dots$ i. $\phi \quad \perp\text{-elimination, } j$	$\frac{\text{Fitch}(j)}{\text{Fitch}(i)} (\perp E)$
j. $\psi \quad \dots$ $\dots$ k. $\phi \quad \dots$ $\dots$ i. $\phi \leftrightarrow \psi \quad \leftrightarrow\text{-introduction, } j, k$	$\frac{\text{Fitch}(j) \quad \text{Fitch}(k)}{\text{Fitch}(i)} (\leftrightarrow I)$
j. $\phi \leftrightarrow \psi \quad \dots$ $\dots$ k. $\phi \quad \dots$ $\dots$ i. $\psi \quad \leftrightarrow\text{-elimination1, } j, k$	$\frac{\text{Fitch}(j) \quad \text{Fitch}(k)}{\text{Fitch}(i)} (\leftrightarrow E_1)$
j. $\phi \leftrightarrow \psi \quad \dots$ $\dots$ k. $\psi \quad \dots$ $\dots$ i. $\phi \quad \leftrightarrow\text{-elimination2, } j, k$	$\frac{\text{Fitch}(j) \quad \text{Fitch}(k)}{\text{Fitch}(i)} (\leftrightarrow E_2)$

j. $\neg\phi$ k. $\perp$ i. ϕ RAA, j-k	$\frac{\text{Fitch}(k)}{\text{Fitch}(i)} \text{ (RAA)}$
j. $\neg\neg\phi$ i. ϕ $\neg\neg$ -elimination, j	$\frac{\text{Fitch}(j)}{\text{Fitch}(i)} (\neg\neg)$
i. $\phi \vee \neg\phi$ TND	$\overline{\vdash \phi \vee \neg\phi} \text{ (TND)}$
j. ψ i. ϕ context, j	$\frac{\text{Fitch}(j)}{\text{Fitch}(i)} \text{ (FromContext)}$
j. ψ i. ϕ contextWeak, j	$\frac{\text{Fitch}(j)}{\text{Fitch}(i)} \text{ (FromWeakenContext)}$

Warto zauważyć, że aby dowody były poprawne, muszą być spełnione odpowiednie warunki dotyczące sekwentów, do których się odwołujemy, wynikające z postaci reguł wnioskowania. Warunki te nie są jednak podane explicite, lecz wynikają z możliwości dopasowania struktury sekwentów do schematów reguł.

Na przykład dla zapisu:

j. $\phi \vee \psi$...
...
k. ρ ...
...
l. ρ ...
...
i. ρ $\vee$ -elimination, j, k, l

który odpowiada dopasowaniu schematu

$$\frac{\text{Fitch}(j) \quad \text{Fitch}(k) \quad \text{Fitch}(l)}{\text{Fitch}(i)} (\vee E)$$

do schematu reguły

$$\frac{\Gamma_1 \vdash \phi \vee \psi \quad \Gamma_2, \phi \vdash \rho \quad \Gamma_3, \psi \vdash \rho}{\Gamma_1, \Gamma_2, \Gamma_3 \vdash \rho} (\vee E)$$

poprawność zastosowania reguły $\forall E$ wymaga, aby teza sekwentu $\text{Fitch}(j)$ była alternatywą, żeby lewa strona tej alternatywy znajdowała się w kontekście sekwentu $\text{Fitch}(k)$, a prawa w kontekście sekwentu $\text{Fitch}(l)$, a także żeby tezy sekwentów $\text{Fitch}(k)$ oraz $\text{Fitch}(l)$ były takie same.

Podobne warunki wynikają dla wszystkich reguł wnioskowania i sprowadzają się do sprawdzenia, czy struktura użytych sekwentów odpowiada instancji danej reguły.

W związku z tym konieczne jest wprowadzenie formalizmu, który pozwala jednoznacznie odtworzyć sekwent odpowiadający każdej linii dowodu, sprawdzić poprawność ich dopasowania do reguł wnioskowania, a także zweryfikować, czy formuła zapisana w danej linii odpowiada tezie kodowanego przez nią sekwentu.

Rozdział 2.

Reprezentacja formuł, sekwentów, reguł wnioskowania i dowodów

Pierwszym etapem konstrukcji systemu było zdefiniowanie reprezentacji obiektów logicznych, na bazie których wykonywane będą dalsze operacje: parsowanie dowodów, sprawdzanie poprawności zastosowań reguł, generowanie danych treningowych oraz weryfikacja kroków proponowanych przez model językowy. Reprezentacja ta musiała spełniać dwa podstawowe wymagania. Po pierwsze, powinna wiernie odpowiadać formalizmowi dedukcji naturalnej opisanemu w poprzednim rozdziale. Po drugie, powinna umożliwiać łatwe przekształcanie obiektów logicznych na postać tekstową, ponieważ właśnie na takich tekstowych reprezentacjach uczony jest model językowy.

2.1. Reprezentacja formuł i sekwentów

Podstawowymi obiektami składniowymi są zmienne, relacje oraz formuły. W obecnej wersji systemu wykorzystywane są jedynie zmienne zdaniowe, jednak reprezentacja została zaprojektowana w sposób nieco ogólniejszy. Atom traktowany jest jako zastosowanie relacji do listy argumentów. W przypadku rachunku zdań wystarcza relacja jednoargumentowa, dlatego zapis taki jak x może być interpretowany jako uproszczona postać atomu. Takie rozwiązanie nie jest konieczne dla samego rachunku zdań, ale ułatwia ewentualne rozszerzenie systemu o bardziej ogólną składnię, na przykład o predykaty wieloargumentowe.

Formuły złożone reprezentowane są rekurencyjnie. Formuła atomowa jest przypadkiem bazowym, natomiast negacja, koniunkcja, alternatywa, implikacja i równoważność przechowują swoje bezpośrednie podformuły. Dzięki temu struktura danych odpowiada drzewiastej strukturze składniowej formuły. Taka reprezentacja pozwala

zarówno łatwo analizować postać formuły, jak i wykonywać operacje potrzebne przy stosowaniu reguł wnioskowania, na przykład odczytanie lewej i prawej strony implikacji czy koniunkcji.

Kontekst dowodzenia reprezentuje zbiór formuł przyjętych jako założenia danego sekwentu. W implementacji został on podzielony na dwie części: kontekst bazowy oraz kontekst dodatkowy. Kontekst bazowy zawiera formuły traktowane jako założenia obowiązujące w całym rozważanym fragmencie dowodu. Kontekst dodatkowy zawiera natomiast formuły wprowadzane lokalnie przez reguły wnioskowania, na przykład przy wprowadzaniu implikacji, negacji albo przy rozumowaniu nie wprost.

Podział ten nie zmienia znaczenia logicznego sekwentu, ale ma znaczenie techniczne i algorytmiczne. Pozwala odróżnić założenia, które powinny być stale dostępne podczas generowania poddowodu, od założeń lokalnych, które mogą być wprowadzane i usuwane przez konkretne reguły. Jest to szczególnie istotne później, gdy system będzie rozbił jeden problem dowodowy na podproblemy oraz generował dowody sekwentów z niepustym kontekstem.

Zdefiniowany wcześniej sekwent jest w systemie reprezentowany jako para złożona z kontekstu oraz tezy. W implementacji obiekt ten nosi nazwę `LittleProblem`, ponieważ każdy sekwent można traktować jako mały problem dowodowy: należy wyprowadzić określoną tezę przy założeniu określonego kontekstu. Taka nazwa jest szczególnie naturalna w dalszej części pracy, gdzie większe problemy dowodowe będą rozbijane na mniejsze podproblemy.

Reprezentacja sekwentu jest bezpośrednio wykorzystywana przez wszystkie dalsze komponenty programu: reguły wnioskowania przekształcają listy sekwentów-przesłanek w sekwenty-konkluzje, parser odtwarza sekwent zakodowane w kolejnych liniach dowodu, a weryfikator sprawdza, czy dana linia rzeczywiście odpowiada sekwentowi otrzymanemu przez zastosowanie wskazanej reguły.

2.2. Reprezentacja reguł wnioskowania

Reguły wnioskowania zostały zaimplementowane jako osobne obiekty odpowiadające regułom systemu przedstawionym w Tabeli 1.1., a także pomocniczym regułom `FromContext` oraz `FromWeakenContext`. Na poziomie programu każda reguła działa przede wszystkim jako funkcja częściowa: dla poprawnej listy sekwentów-przesłanek zwraca sekwent będący konkluzją zastosowania tej reguły, natomiast dla danych, które nie pasują do jej schematu, zgłasza błąd.

Można to opisać następującym schematem:

Algorithm 1 Zastosowanie reguły wnioskowania

```

1: procedure ZASTOSUJREGULĘ( $R, P, A$ )
2:   if  $P$  nie spełnia warunków wejściowych reguły  $R$  then
3:     zgłoś błąd
4:   end if
5:    $S \leftarrow$  konkluzja reguły  $R$  dla przesłanek  $P$  i argumentów  $A$ 
6:   return  $S$ 
7: end procedure

```

Warunki wejściowe obejmują zgodność liczby przesłanek, zgodności ich typów (każda przesłanka musi być sekwentem), oraz ich zgodności ze schematem danej reguły. Argument R oznacza regułę wnioskowania, P oznacza listę sekwentów-przesłanek, natomiast A oznacza dodatkowe dane potrzebne w tych regułach, których zastosowanie nie jest jednoznacznie wyznaczone przez same przesłanki. Przykładowo w regule $(\forall I_1)$ z samej przesłanki można odczytać jedynie formułę φ , ponieważ przesłanka ma postać $\Gamma \vdash \varphi$. Aby skonstruować konkluzję

$$\Gamma \vdash \varphi \vee \psi,$$

system musi dodatkowo znać formułę ψ . Nie jest ona wyznaczona przez przesłankę, dlatego jest przekazywana jako dodatkowy argument zastosowania reguły. Analogiczna sytuacja występuje w regule $(\forall I_2)$, gdzie dodatkowym argumentem jest lewy składnik alternatywy.

Istotne jest to, że niepoprawne zastosowanie reguły nie jest reprezentowane przez wartość logiczną **False**. Zamiast tego reguła zgłasza błąd. Jest to wygodne z punktu widzenia dalszej konstrukcji systemu, ponieważ zastosowanie reguły można traktować jako operację, która albo zwraca poprawnie skonstruowany sekwent, albo w ogóle nie jest zdefiniowana dla podanych argumentów. Dzięki temu w pozostałych częściach programu nie trzeba po każdym zastosowaniu reguły osobno sprawdzać, czy otrzymany wynik jest poprawny. Błędne kandydatury mogą być po prostu odrzucane przez przechwycenie błędu.

2.3. Reprezentacja dowodów

W poprzednim rozdziale przyjęliśmy, że każda linia dowodu koduje jeden sekwent. Na pierwszy rzut oka mogłoby się więc wydawać, że po sparsowaniu dowodu wystarczy przechowywać wyłącznie sekwenty odpowiadające kolejnym liniom. Takie rozwiązanie pozwalałoby odtworzyć, jakie sekwenty zostały kolejno otrzymane, ale traciłoby informację o tym, jak poszczególne kroki zostały uzasadnione.

Sama reprezentacja sekwentu jest więc zbyt uboga jako reprezentacja linii dowodu. Sekwent jest logicznym rezultatem danej linii, natomiast linia zawiera również

dane potrzebne do odtworzenia tego rezultatu: numer linii, formułę zapisaną jako jej tezę, użytą regułę wnioskowania oraz indeksy występujące w uzasadnieniu. Bez tych informacji sparsowany dowód byłby jedynie ciągiem otrzymanych sekwentów, a nie strukturą pozwalającą odtworzyć zależności między krokami.

Sparsowana linia dowodu nie przechowuje oryginalnego tekstu linii, lecz informacje wydobyte z tego tekstu oraz sekwent odtworzony na ich podstawie. Do informacji odczytanych z zapisu należą: numer linii, lista indeksów występujących w jej uzasadnieniu, obiekt reprezentujący zastosowaną regułę oraz formuła zapisana jako teza linii. Na podstawie tych danych, a także wcześniej sparsowanej części dowodu, konstruowany jest sekwent kodowany przez daną linię.

Można to przedstawić schematycznie w następujący sposób:

$$\text{zapis linii} \longrightarrow (i, \varphi, R, J) \longrightarrow S_i.$$

Tutaj i oznacza numer aktualnej linii, φ formułę zapisaną w tej linii, R zastosowaną regułę, natomiast $J = (j_1, \dots, j_n)$ oznacza ciąg indeksów podanych w uzasadnieniu linii. Indeksy te nie muszą być dokładnie listą przesłanek reguły wnioskowania, ale w zależności od reguły mogą również zawierać indeksy linii z których czerpiemy dodatkowe argumenty do wywołania tej reguły, które nie są przesłankami. Na przykład linia postaci

$$\text{i. } \phi \quad \text{weakening, j, k}$$

zawiera dwa indeksy, ale tylko jeden z nich wskazuje właściwą przesłankę reguły. Sekwent S_k jest sekwentem, który ma zostać osłabiony, natomiast linia j służy do odczytania formuły dodawanej do kontekstu. Jeśli więc $S_k = \Gamma \vdash \phi$, a tezą linii j jest ψ , to system konstruuje sekwent

$$\Gamma, \psi \vdash \phi.$$

W tym przypadku dodatkowa formuła ψ nie pochodzi z formuły zapisanej w aktualnej linii, lecz z tezy wcześniejszej linii wskazanej przez jeden z indeksów w J .

Innym przykładem jest linia postaci

$$\text{i. } \phi \vee \psi \quad \vee\text{-introduction1, j}$$

Tutaj indeks j wskazuje właściwą przesłankę, czyli wcześniejszy sekwent $S_j = \Gamma \vdash \varphi$. Sama przesłanka nie wyznacza jednak formuły ψ , która ma zostać dodana jako prawa strona alternatywy. System odczytuje ją więc z formuły zapisanej w aktualnej linii, czyli z prawego składnika $\varphi \vee \psi$, i przekazuje ją jako dodatkowy argument reguły.

Jeszcze innym przypadkiem są reguły, które nie odwołują się do żadnej wcześniejszej linii. Przykładem jest linia postaci

$$\text{1. } \phi \quad \text{assumption}$$

W najprostszym przypadku odpowiada ona sekwentowi $\phi \vdash \phi$. W naszym systemie sekwent konstruowany dla takiej linii może jednak zawierać również niepusty kontekst bazowy, obowiązujący w całym aktualnie rozważanym fragmencie dowodu. Tego kontekstu nie da się odczytać z wcześniejszych sparsowanych linii, ponieważ takowe jeszcze nie istnieją. Dlatego kontekst bazowy musi zostać przekazany *explicite* jako dodatkowy argument podczas konstruowania linii.

Sparsowany dowód jest przechowywany jako słownik, który mapuje indeksy linii na odpowiadające im sparsowane linie. Jeżeli przez D oznaczymy taki słownik, to $D[i]$ jest obiektem reprezentującym linię o numerze i , a $D[i].LP$ oznacza sekwent odtworzony dla tej linii. Dzięki temu podczas parsowania kolejnej linii wystarczy odwołać się do indeksów zapisanych w jej uzasadnieniu i pobrać z D potrzebne wcześniej skonstruowane obiekty.

Schematycznie parsowanie dowodu można więc opisać następująco:

$$D_0 = \emptyset,$$

$$D_i = D_{i-1} \cup \{i \mapsto L_i\},$$

gdzie L_i jest sparsowaną postacią i -tej linii dowodu, skonstruowaną przy użyciu słownika D_{i-1} . Oznacza to, że każda kolejna linia może korzystać wyłącznie z informacji odtworzonych dla wcześniejszych linii. Taki sposób parsowania odpowiada liniowej strukturze dowodu: późniejsze kroki mogą odwoływać się do wcześniejszych, ale nie odwrotnie.

Po skonstruowaniu sparsowanej linii system sprawdza jeszcze, czy teza odtworzonego sekwentu jest identyczna z formułą zapisaną w tekście linii. Ten krok jest konieczny, ponieważ formuła występująca w zapisie linii jest traktowana jako deklaracja, a nie jako samodzielne źródło informacji o całym sekwencie. Jeżeli reguła zastosowana do odtworzonych przesłanek i argumentów prowadzi do innej tezy niż ta zapisana w linii, linia zostaje odrzucona. W ten sposób parser pełni jednocześnie rolę weryfikatora poprawności lokalnych kroków dowodu.

Niepoprawna linia nie jest oznaczana specjalną wartością logiczną, lecz powoduje zgłoszenie błędu. Dotyczy to zarówno błędów składniowych, jak i sytuacji, w których indeksy zapisane w uzasadnieniu nie pozwalają skonstruować poprawnego zastosowania wskazanej reguły. Takie rozwiązanie upraszcza późniejsze etapy działania systemu. Podczas generowania korpusu lub sprawdzania linii proponowanych przez model językowy można po prostu próbować skonstruować sparsowaną linię; jeśli konstrukcja się powiedzie, linia jest poprawna, a jeśli pojawi się błąd, kandydata zostaje odrzucona.

Ta sama reprezentacja jest wykorzystywana przy generowaniu korpusu. Program nie tworzy linii treningowych przez ręczne sklejanie napisów, lecz najpierw konstruuje poprawny obiekt linii, a dopiero potem wypisuje go w ustalonym formacie tekstowym. Dzięki temu każda wygenerowana linia ma od razu odpowiadającą jej strukturę logiczną, którą parser potrafi ponownie odtworzyć.

Rozdział 3.

Strategie konstruowania dowodów

3.1. Motywacja

Celem systemu nie jest jedynie wygenerowanie tekstu przypominającego dowód w dedukcji naturalnej, lecz znalezienie poprawnego dowodu danej tautologii. Model językowy pełni w tym procesie rolę generatora kandydatów: proponuje kolejne linie dowodu albo inne kroki pomocnicze, ale nie decyduje samodzielnie o ich poprawności. Każdy wygenerowany fragment musi zostać zweryfikowany przez opisany wcześniej parser i mechanizm sprawdzania reguł wnioskowania.

Takie rozdzielenie ról jest kluczowe. Model językowy może dobrze naśladować składnię dowodów i często proponować sensowne kroki, ale sam z siebie nie gwarantuje poprawności logicznej. Może wygenerować linię o poprawnym wyglądzie, która jednak odwołuje się do niewłaściwych wcześniejszych linii, używa reguły w niepoprawny sposób albo wprowadza formułę, której nie da się uzasadnić w danym kontekście. Dlatego w systemie model nie jest traktowany jako samodzielny system, lecz jako źródło propozycji filtrowanych przez formalny weryfikator.

Najprostsze podejście polega na liniowym dopisywaniu kolejnych linii dowodu. System przekazuje modelowi cel dowodowy oraz dotychczas wygenerowany fragment dowodu, a następnie próbuje dopisać jedną nową linię. Jeżeli po jej dopisaniu dowód nadal poprawnie się parsuje, linia zostaje zaakceptowana; w przeciwnym razie jest odrzucana. Proces generacji trwa do momentu uzyskania linii kodującej poszukiwany sekwent albo do chwili, w której dalsze rozszerzanie dowodu zostaje uznane za niecelowe. Tak powstaje pierwsza strategia, w której dowód konstruowany jest jako jedna rosnąca sekwencja lokalnie poprawnych kroków.

Podejście to ma jednak istotne ograniczenie. Konstruowanie dowodu rzadko jest procesem całkowicie liniowym. Nawet jeśli każda kolejna linia jest poprawna, może

okazać się, że wybrana ścieżka nie prowadzi do celu albo prowadzi do niego bardzo okreżną drogą. W praktyce potrzebne jest więc pewne przeszukiwanie przestrzeni możliwych częściowych dowodów: system powinien móc rozwijać kilka alternatywnych prób i wracać do bardziej obiecujących fragmentów. Tę ideę realizuje druga strategia, która przechowuje wiele częściowych dowodów i wybiera spośród nich te, które warto dalej rozwijać.

Dla trudniejszych formuł nawet takie lokalne przeszukiwanie może być niewystarczające. Problemem nie jest wtedy tylko wybór następnej linii, lecz wybór ogólnego planu dowodu. Często łatwiej jest najpierw zdecydować, jaką regułą można byłoby otrzymać cel, a następnie spróbować udowodnić jej przesłanki jako osobne podproblemy. W ten sposób otrzymujemy trzecią strategię, w której system może rozbić początkowy problem na prostsze sekwenty, a następnie szukać dowodów dla nich. Strategia ta łączy generowanie dowodów w notacji liniowej z przeszukiwaniem drzewa problemów i podproblemów.

Kolejne strategie powstały więc jako odpowiedź na ograniczenia poprzednich. Pierwsza strategia sprawdza, czy model potrafi lokalnie dopisywać poprawne linie dowodu. Druga dodaje mechanizm wyboru między wieloma częściowymi próbami, ale nadal wykorzystuje pierwszą strategię jako procedurę rozwijania pojedynczego fragmentu dowodu. Trzecia zmienia poziom poszukiwania: oprócz dopisywania linii pozwala planować dowód przez dekompozycję celu na podproblemy. Również ona nie zastępuje wcześniejszych metod, lecz korzysta z nich jako komponentów pomocniczych: po rozbiciu celu na podproblemy próbujemy znaleźć dowody tych podproblemów przy użyciu drugiej strategii.

W ten sposób strategie tworzą hierarchię. Każda kolejna strategia rozszerza poprzednią, traktując ją jako niezależny komponent realizujący prostsze zadanie. Dzięki temu system można opisywać warstwowo: najpierw jako generator pojedynczej poprawnej kontynuacji dowodu, następnie jako mechanizm przeszukiwania wielu takich kontynuacji, a ostatecznie jako procedurę rozbijania problemu dowodowego na mniejsze problemy, dla których ponownie uruchamiane są wcześniejsze mechanizmy. W dalszej części rozdziału strategie te zostaną opisane w tej kolejności.

3.2. Strategia pierwsza: liniowe dopisywanie kroków dowodu

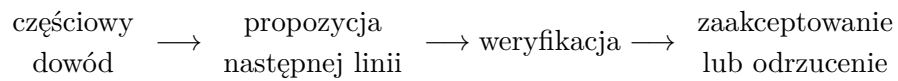
3.2.1. Idea strategii

Pierwsza strategia jest najprostszym sposobem wykorzystania modelu językowego w systemie dowodzącym. Jej celem jest sprawdzenie, czy model potrafi generować kolejne poprawne kroki dowodu w notacji liniowej, jeżeli otrzyma opis celu oraz dotychczasowy fragment dowodu. Strategia ta nie próbuje planować całego do-

wodu z góry ani nie rozważa wielu alternatywnych ścieżek. Dowód jest konstruowany sekwencyjnie: w każdym kroku system próbuje dopisać jedną nową linię do końca aktualnego fragmentu.

W tym sensie strategia pierwsza traktuje model jako lokalny generator kontynuacji. Wejściem dla modelu jest prompt kodujący sekwent, który ma zostać udowodniony, oraz ewentualnie dotychczas wygenerowane linie dowodu. Wyjściem nie jest cały dowód, lecz kandydatura na następną linię. Poprawność tej kandydatury nie jest oceniana przez model, lecz przez formalny parser i mechanizm sprawdzania reguł wnioskowania.

Schematycznie można to przedstawić następująco:



W implementacji tej strategii odpowiada podstawowa procedura lokalnego rozwijania częściowego dowodu, oznaczona jako `silliest_generate_proof`. Nazwa ta odzwierciedla prototypowy charakter pierwszej wersji algorytmu: strategia jest celowo prosta i służy jako punkt wyjścia dla bardziej złożonych metod opisanych w dalszej części rozdziału.

3.2.2. Generowanie i weryfikacja pojedynczej linii

Podstawowa operacja wykonywana przez strategię polega na próbie dopisania jednej nowej linii do aktualnego fragmentu dowodu. Nie odbywa się to jednak przez zupełnie swobodne wygenerowanie dowolnego ciągu znaków przez model. System narzuca częściową strukturę generowanej linii, tak aby model odpowiadał przede wszystkim za wybór formuły i uzasadnienia, a nie za odtwarzanie elementów czysto technicznych, takich jak numeracja linii.

Najpierw analizowany jest aktualny prompt. Jeżeli zawiera on wyłącznie cel dowodowy, następna linia otrzymuje numer 1. Jeżeli w prompcie znajdują się już wcześniejsze linie dowodu, system odczytuje numer ostatniej z nich i rozpoczyna nową linię od kolejnego numeru. Dzięki temu model nie musi samodzielnie utrzymywać poprawnej numeracji dowodu, co zmniejsza liczbę błędów czysto składniowych.

Następnie generowana jest formuła zapisana w nowej linii. Na tym etapie system ogranicza zbiór akceptowanych tokenów. Dopuszczalne są symbole logiczne, nawiasy, stałe $\top$ i $\perp$, spacje oraz zmienne występujące w dowodzonym sekwencie. Jeżeli dowód ma niepusty kontekst bazowy, zbiór dopuszczalnych zmiennych jest wyznaczany na podstawie zarówno tezy, jak i formuł występujących w kontekście. Celem tego ograniczenia jest uniknięcie sytuacji, w której model wprowadza do dowodu nowe, przypadkowe zmienne, niezwiązane z rozważanym problemem.

Generowanie formuły trwa do momentu, w którym pojawi się separator oddzielający formułę od nazwy reguły uzasadniającej linię. W przyjętym formacie rolę tę

pełnią cztery kolejne spacje. Po ich wygenerowaniu system przechodzi do drugiej części linii, czyli do uzasadnienia zawierającego nazwę reguły oraz ewentualne indeksy wcześniejszych linii. Ta część jest generowana już jako kontynuacja rozpoczętej linii i kończy się w momencie wygenerowania znaku nowej linii.

Można to opisać schematycznie następująco:

prompt $\longrightarrow$ numer nowej linii $\longrightarrow$ formuła $\longrightarrow$ uzasadnienie.

W implementacji ten etap odpowiada procedurze generowania następnej linii, która na podstawie aktualnego prompta tworzy kandydaturę na kolejną linię dowodu.

Po wygenerowaniu kandydackiej linii system dopisuje ją tymczasowo do aktualnego fragmentu dowodu i próbuje ponownie sparsować całość. Sprawdzenie to nie ogranicza się do kontroli składni. Parser próbuje odtworzyć strukturę każdej linii, skonstruować odpowiadające jej sekwenty i zweryfikować, czy zastosowane reguły rzeczywiście prowadzą do zapisanych tez. Jeżeli parsowanie kończy się błędem, wygenerowana linia zostaje odrzucona.

Kandydacka linia może zostać odrzucona z kilku powodów. Po pierwsze, może nie spełniać wymagań składniowych przyjętego formatu. Po drugie, może zawierać formułę niezgodną z ograniczeniami na zmienne występujące w dowodzonym problemie (sprawdzenie tego warunku jest defacto nadmiarowe, ponieważ jest on sprawdzany już na etapie generowania linii). Po trzecie, może wyglądać jak poprawna linia dowodu, ale odwoływać się do wcześniejszych linii w sposób niezgodny ze schematem wskazanej reguły. W każdym z tych przypadków parser lub weryfikator zgłasza błąd, a wygenerowana kandydatura nie jest dopisywana do dowodu.

Jeżeli natomiast dopisanie linii nie powoduje błędu parsowania, linia zostaje zaakceptowana. Oznacza to, że udało się odtworzyć sekwent kodowany przez tę linię, a jego teza zgadza się z formułą zapisaną w tekście. Po zaakceptowaniu linii system traktuje ją tak samo jak wcześniejsze linie: może ona zostać wykorzystana jako uzasadnienie kolejnych kroków dowodu.

W praktyce strategia działa więc jako pętla generowania i filtrowania. Jeśli przez D_k oznaczmy aktualny częściowy dowód, a przez ℓ kandydaturę na nową linię, to po jej tymczasowym dopisaniu otrzymujemy

$$D_{k+1}^* = D_k + \ell.$$

Następnie system wykonuje weryfikację:

$$D_{k+1} = \begin{cases} D_{k+1}^*, & \text{jeśli } D_{k+1}^* \text{ jest poprawny,} \\ D_k, & \text{w przeciwnym przypadku.} \end{cases}$$

Takie rozwiązanie pozwala oddzielić zadanie generowania od zadania sprawdzania. Model językowy odpowiada za proponowanie potencjalnie sensownych kontynuacji, natomiast poprawność logiczna pozostaje kontrolowana przez formalny mechanizm opisany w poprzednim rozdziale.

3.2.3. Warunki zakończenia

Strategia kończy działanie sukcesem wtedy, gdy ostatnia zaakceptowana linia koduje sekwent docelowy. Innymi słowy, po sparsowaniu aktualnego fragmentu dowodu system sprawdza, czy sekwent odtworzony dla ostatniej linii jest równy sekwentowi, który miał zostać udowodniony. Jeśli tak jest, wygenerowany tekst stanowi pełny dowód celu.

Działanie strategii może zakończyć się także bez znalezienia dowodu. Pierwszym ograniczeniem jest maksymalna liczba nowych linii, które wolno dopisać do prompta. Chroni to system przed generowaniem bardzo długich fragmentów, które lokalnie pozostają poprawne, ale nie zbliżają się do celu. Drugim ograniczeniem jest maksymalna liczba kolejnych nieudanych prób dopisania linii. Jeżeli model przez wiele prób z rzędu generuje kandydatury odrzucone przez parser, system uznaje, że dalsze kontynuowanie tej samej ścieżki jest mało obiecujące.

W przypadku zakończenia bez sukcesu strategia może mimo to zwrócić częściowy dowód, czyli prompt z dopisanymi poprawnymi liniami. Taki fragment nie musi jeszcze dowodzić celu, ale może stanowić postęp względem punktu wyjścia. Właśnie dlatego pierwsza strategia może być później użyta jako komponent w strategii drugiej: nie musi samodzielnie zawsze znaleźć pełnego dowodu, wystarczy, że potrafi lokalnie rozwijać częściowy dowód o kolejne poprawne kroki.

3.2.4. Ograniczenia strategii

Największym ograniczeniem pierwszej strategii jest jej liniowy charakter. System rozwija tylko jedną ścieżkę dowodu i nie przechowuje alternatywnych kontynuacji. Jeżeli model wybierze poprawną, ale mało użyteczną linię, strategia nie ma mechanizmu powrotu do wcześniejszego stanu i wyboru innej możliwości. Może więc utknąć w ciągu kroków, które są lokalnie poprawne, ale nie prowadzą do dowodu celu.

Drugim ograniczeniem jest brak globalnego planowania. Strategia nie analizuje struktury celu w taki sposób, aby zdecydować, jakie podcele warto najpierw osiągnąć. Każdy krok jest generowany na podstawie dotychczasowego tekstu dowodu, ale system nie rozważa jawnie, jaką regułą można byłoby zakończyć dowód ani jakie przesłanki należałoby wcześniej udowodnić. W prostych przypadkach lokalne dopisywanie linii może wystarczyć, ale dla bardziej złożonych formuł często potrzebne jest planowanie na wyższym poziomie.

Skuteczność tej strategii zależy również od parametrów generacji, takich jak temperatura oraz liczba dopuszczalnych prób wygenerowania kolejnej linii. Nie jest to jednak osobny problem konstrukcyjny, lecz ogólna konsekwencja użycia modelu językowego jako generatora kandydatów.

Mimo tych ograniczeń strategia pierwsza jest ważnym elementem całego systemu. Stanowi podstawowy mechanizm lokalnego rozszerzania dowodu, który może być używany jako komponent w bardziej złożonych metodach. Druga strategia zachowuje ten mechanizm generowania pojedynczych kroków, ale dodaje możliwość rozwijania wielu częściowych dowodów równoległe, co częściowo rozwiązuje problem utknięcia w jednej niekorzystnej ścieżce.

3.3. Strategia druga: przeszukiwanie wielu częściowych dowodów

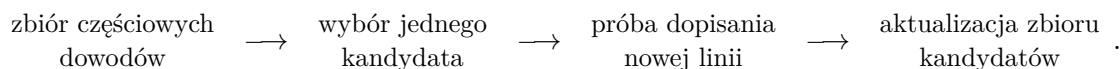
3.3.1. Motywacja

Pierwsza strategia konstruuje tylko jeden częściowy dowód. Oznacza to, że każda zaakceptowana linia staje się trwałą częścią aktualnej próby. Jeżeli później okaże się, że dana kontynuacja nie prowadzi w stronę rozwiązania, system nie ma możliwości powrotu do wcześniejszego etapu i wyboru innej ścieżki rozumowania. W praktyce jest to poważne ograniczenie, ponieważ poprawność lokalna pojedynczej linii nie gwarantuje jeszcze, że linia ta będzie użyteczna dla całego dowodu.

Druga strategia próbuje ograniczyć ten problem przez jednoczesne przechowywanie wielu częściowych dowodów. Zamiast rozwijać wyłącznie jedną sekwencję linii, system utrzymuje zbiór kandydatów. W każdej iteracji wybierany jest jeden częściowy dowód, a następnie podejmowana jest próba dopisania do niego kolejnej poprawnej linii. Jeżeli powstały w ten sposób fragment jest poprawny i nie wystąpił wcześniej, zostaje dodany do zbioru kandydatów.

Można więc powiedzieć, że druga strategia zmienia charakter generowania dowodu z liniowego dopisywania kolejnych kroków na przeszukiwanie przestrzeni częściowych dowodów. Pojedynczy nieoptymalny wybór nie przekreśla całego procesu, ponieważ inne częściowe dowody pozostają dostępne i mogą zostać rozwinięte w późniejszych iteracjach.

Schematycznie można to przedstawić następująco:



3.3.2. Zbiór częściowych dowodów

Podstawową strukturą wykorzystywaną przez drugą strategię jest zbiór częściowych dowodów. Na początku zawiera on jedynie prompt opisujący sekwent, który należy udowodnić. W kolejnych iteracjach do zbioru dodawane są nowe częściowe dowody powstałe przez dopisanie poprawnej linii do jednego z wcześniej istniejących kandydatów.

Każdy element tego zbioru jest tekstem zawierającym opis celu dowodowego oraz dotychczas wygenerowany fragment dowodu. Dzięki temu może zostać bezpośrednio podany modelowi językowemu jako kontekst generacji.

W każdej iteracji system wykonuje następujące kroki:

Algorithm 2 Rozwijanie zbioru częściowych dowodów

- 1: Wybierz częściowy dowód D ze zbioru kandydatów P .
 - 2: Spróbuj dopisać do D jedną nową poprawną linię, otrzymując D' .
 - 3: Sprawdź poprawność otrzymanego fragmentu D' .
 - 4: **if** D' kończy dowód celu **then**
 - 5: **return** D'
 - 6: **else if** D' jest poprawnym nowym częściowym dowodem **then**
 - 7: Dodaj D' do zbioru kandydatów P .
 - 8: **end if**
-

Ważne jest, że druga strategia nie zastępuje mechanizmu generowania pojedynczej linii, lecz używa go jako podprocedury. Próba rozwinięcia częściowego dowodu nadal polega na tym, że model proponuje kandydacką linię, a formalny parser i weryfikator sprawdzają jej poprawność. Różnica polega na tym, że poprawne rozwinięcia nie muszą tworzyć jednej liniowej historii. Mogą współistnieć jako alternatywne ścieżki poszukiwania dowodu.

3.3.3. Ocena prawdopodobieństwa częściowego dowodu

Jeżeli zbiór częściowych dowodów zawiera wiele kandydatów, trzeba zdecydować, który z nich rozwijać w następnej iteracji. Najprostsze rozwiązanie polegałoby na losowaniu jednostajnym, jednak ignorowałoby ono to, że nie wszystkie częściowe dowody są tak samo prawdopodobne. Model nie tylko generuje tekst, ale również przypisuje kolejnym tokenom prawdopodobieństwa. Można więc użyć tych prawdopodobieństw jako przybliżonej oceny tego, jak naturalny dla modelu jest dany częściowy dowód.

Dla każdego częściowego dowodu obliczana jest wartość zależna od prawdopodobieństw tokenów tworzących wygenerowaną część dowodu. Ponieważ iloczyn wielu prawdopodobieństw bardzo szybko staje się liczbą bliską zeru, wygodniej sumować logarytmy prawdopodobieństw. Aby nie faworyzować nadmiernie krótkich fragmentów tylko dlatego, że składają się z mniejszej liczby tokenów, suma ta jest normalizowana przez długość analizowanego tekstu.

Jeżeli częściowy dowód oznaczmy przez D , a jego kolejne tokeny przez $t_1, \dots, t_m$, to jego ocenę można zapisać schematycznie jako

$$\text{score}(D) = \frac{1}{m} \sum_{i=1}^m \log P(t_i \mid t_1, \dots, t_{i-1}, \text{prompt celu}).$$

Wartość $score(D)$ jest jednak nieokreślona, gdy $m = 0$, czyli dla początkowego prompta, który nie zawiera jeszcze żadnych wygenerowanych tokenów. W takim przypadku najpierw wyliczamy $score$ dla pozostałych częściowych dowodów, a następnie ustawiamy $score$ dla pustego prompta na wartość średnią z tych ocen. Intuicja jest taka, że początkowy prompt nie powinien być ani stale faworyzowany, ani całkowicie pomijany. Z jednej strony nie zawiera on jeszcze żadnego postępu w konstrukcji dowodu, więc zbyt częste wybieranie go prowadziłoby do wielokrotnego rozpoczynania generacji od początku. Z drugiej strony jest to kandydat najbardziej neutralny: nie zawiera żadnych wcześniejszych decyzji, które mogłyby skierować dowód w mało użyteczną stronę. Przypisanie mu średniej oceny pozwala więc zachować możliwość powrotu do początku rozumowania, ale nie czyni z niego domyślnie najatrakcyjniejszej ścieżki.

Kolejnym problemem jest to, że wartości $score(D)$, jako średnie logarytmów prawdopodobieństw, są w praktyce niedodatnie. Mogą być równe 0 jedynie w granicznym przypadku, gdy każdy kolejny token ma prawdopodobieństwo warunkowe równe 1. Ponadto wartości te nie sumują się do 1, a więc nie mogą być bezpośrednio interpretowane jako rozkład prawdopodobieństwa. Aby przekształcić je w rozkład na zbiorze kandydatów, używamy funkcji softmax:

$$P(D_i) = \frac{\exp(score(D_i))}{\sum_j \exp(score(D_j))}.$$

W ten sposób każdemu częściowemu dowodowi przypisana zostaje dodatnia wartość, a suma wszystkich takich wartości wynosi 1. Kandydaci o wyższym wyniku $score$, czyli bardziej prawdopodobni według modelu, otrzymują większe prawdopodobieństwo wyboru w następnej iteracji.

3.3.4. Pełny przebieg strategii

Po wprowadzeniu zbioru częściowych dowodów oraz sposobu przypisywania im prawdopodobieństw wyboru można przedstawić całą drugą strategię jako jedną pętlę przeszukiwania. W każdej iteracji system aktualizuje oceny kandydatów, wybiera jeden z częściowych dowodów zgodnie z rozkładem otrzymanym po zastosowaniu funkcji softmax, a następnie próbuje rozwinąć go o jedną poprawną linię.

Algorithm 3 Przeszukiwanie wielu częściowych dowodów

```

1:  $P \leftarrow \{\text{prompt opisujący cel } G\}$ 
2: while nie przekroczono limitu iteracji do
3:   Oblicz wartości  $\text{score}(D)$  dla kandydatów  $D \in P$ .
4:   Przekształć wartości  $\text{score}(D)$  w rozkład prawdopodobieństwa na  $P$ .
5:   Wylosuj częściowy dowód  $D \in P$  zgodnie z otrzymanym rozkładem.
6:   Spróbuj dopisać do  $D$  jedną poprawną linię, otrzymując  $D'$ .
7:   if  $D'$  jest pełnym dowodem celu  $G$  then
8:     return  $D'$ 
9:   else if  $D'$  jest poprawnym nowym częściowym dowodem then
10:     $P \leftarrow P \cup \{D'\}$ 
11:   end if
12: end while
13: return informacja o niepowodzeniu

```

W powyższym schemacie etap dopisania nowej linii korzysta z procedury opisanej przy pierwszej strategii. Druga strategia nie zmienia więc sposobu weryfikacji pojedynczego kroku dowodu, lecz zmienia organizację całego procesu poszukiwania: zamiast jednej rozwijanej sekwencji utrzymywany jest zbiór alternatywnych prób, spośród których kolejne są wybierane probabilistycznie.

Niepowodzenie algorytmu oznacza jedynie, że dowód nie został znaleziony w ramach przyjętych ograniczeń obliczeniowych. Nie jest to informacja o niedowodliwości danego sekwentu.

3.3.5. Ograniczenia strategii

Druga strategia usuwa najważniejsze ograniczenie strategii liniowej: system nie jest już przywiązany do jednej ścieżki generacji. Może utrzymywać wiele alternatywnych częściowych dowodów i stopniowo rozwijać te, które wydają się bardziej obiecujące. Nie rozwiązuje to jednak wszystkich problemów związanych z automatycznym konstruowaniem dowodów.

Po pierwsze, strategia nadal ma charakter lokalny. Rozwijanie częściowego dowodu polega na dopisaniu kolejnej linii, a nie na świadomym zaplanowaniu całej struktury dowodu. System może więc generować poprawne, ale mało użyteczne kroki, które zwiększają długość dowodu bez realnego przybliżania go do celu.

Po drugie, ocena prawdopodobieństwa częściowego dowodu nie jest tym samym co ocena jego matematycznej przydatności. Model może wysoko oceniać fragmenty podobne do danych treningowych, ale nie oznacza to jeszcze, że są one najlepszą drogą do udowodnienia konkretnej tezy. Prawdopodobieństwo generacji jest więc heurystyką, a nie kryterium poprawności ani gwarancją sukcesu.

Po trzecie, liczba możliwych częściowych dowodów rośnie bardzo szybko. Każda zaakceptowana linia może prowadzić do kolejnych rozwinięć, a wiele z nich może być formalnie poprawnych. W rezultacie strategia wymaga limitów liczby iteracji oraz mechanizmów wyboru kandydatów do rozwijania. Limity te są konieczne obliczeniowo, ale sprawiają, że przeszukiwanie pozostaje niepełne.

Najważniejsze ograniczenie polega jednak na tym, że druga strategia nadal nie wykonuje dekompozycji celu na podcele. System próbuje znaleźć dowód przez rozszerzanie częściowych tekstowych dowodów, ale nie ma osobnego etapu, w którym analizuje strukturę dowodzonego sekwentu i wybiera regułę pozwalającą rozbić go na prostsze problemy. W trudniejszych przypadkach właśnie taka dekompozycja może być bardziej naturalna niż dopisywanie kolejnych linii od początku dowodu.

To prowadzi do trzeciej strategii, w której system może najpierw rozbić dowodzony sekwent na listę prostszych podproblemów, a następnie próbować udowodnić każdy z nich osobno.

3.4. Strategia trzecia: dekompozycja celu i drzewo poszukiwania

3.4.1. Motywacja

Druga strategia pozwala prowadzić wiele częściowych dowodów równolegle, dzięki czemu system nie jest przywiązany do jednej liniowej ścieżki generacji. Nadal jednak rozwijanie dowodu odbywa się przez dopisywanie kolejnych linii do już istniejącego fragmentu. System wybiera więc pewien częściowy dowód i próbuje znaleźć jego następny poprawny krok. Nie analizuje natomiast osobno, jaka ogólna struktura dowodu byłaby potrzebna, aby otrzymać zadany sekwent.

Taki sposób działania odpowiada naturalnemu zapisowi dowodu w notacji Fitcha: zaczynamy od założeń, aksjomatów lub wcześniej uzyskanych sekwentów, a następnie stosujemy reguły wnioskowania, aby dojść do tezy. Jest to konstruowanie dowodu w kierunku od przesłanek do celu. W wielu sytuacjach wygodny jest jednak kierunek przeciwny. Zamiast pytać, jaką kolejną linię można dopisać do częściowego dowodu, można zacząć od sekwentu, który chcemy udowodnić, i zapytać, jakie prostsze sekwenty wystarczyłoby udowodnić, aby otrzymać go przez zastosowanie jednej z reguł wnioskowania.

Można więc patrzeć na konstrukcję dowodu z dwóch stron. Pierwsza strategia buduje dowód od wcześniejszych linii ku końcowej tezie. Trzecia strategia zaczyna od celu i próbuje rozłożyć go na podcele, które następnie mogą być dowodzone osobno. W notacji drzewiastej odpowiada to budowaniu drzewa dowodu od korzenia ku liściom: w korzeniu znajduje się sekwent, który chcemy udowodnić, a kolejne poziomy drzewa zawierają przesłanki reguł, których udowodnienie wystarczy do otrzymania

sekwentów znajdujących się wyżej.

Przykładowo, jeżeli celem jest udowodnienie koniunkcji, naturalnym krokiem jest próba udowodnienia osobno jej dwóch składników. Jeżeli celem jest udowodnienie implikacji, można przyjąć jej poprzednik jako dodatkowe założenie i następnie spróbować udowodnić jej następnik. W obu przypadkach nie zaczynamy od pytania o pierwszą linię dowodu, lecz od pytania o to, jakie podproblemy prowadzą do rozwiązania problemu wyjściowego.

Trzecia strategia rozszerza więc wcześniejsze podejścia o możliwość dekompozycji celu. System może najpierw rozbić dowodzony sekwent na listę prostszych podproblemów, a dopiero potem próbować udowodnić każdy z nich za pomocą wcześniejszych metod generowania dowodów. Jeżeli wszystkie podproblemy danego rozbicia zostaną rozwiązane, to rozwiązany zostaje również problem wyjściowy.

3.4.2. Rozbicie problemu na podproblemy

Rozbiciem problemu na podproblemy nazywamy wskazanie takiej reguły wnioskowania oraz takiej listy sekwentów-przesłanek, że sekwent wyjściowy można otrzymać przez zastosowanie tej reguły do tych przesłanek.

Aby zapisać to precyzyjniej, dla każdej reguły R oraz listy sekwentów $S_1, \dots, S_n$ wprowadzamy oznaczenie

$$R(S_1, \dots, S_n),$$

które oznacza zbiór wszystkich sekwentów mogących wystąpić jako konkluzje poprawnego zastosowania reguły R do przesłanek $S_1, \dots, S_n$.

Jeżeli więc chcemy udowodnić sekwent S , to szukamy reguły R oraz sekwentów $S_1, \dots, S_n$, dla których zachodzi

$$S \in R(S_1, \dots, S_n).$$

Wtedy zadanie udowodnienia S można zastąpić zadaniami udowodnienia $S_1, \dots, S_n$.

Takie rozbicie można interpretować jako jeden krok budowania drzewa dowodu w kierunku przeciwnym do kierunku zapisu dowodu w notacji Fitcha. W notacji Fitcha zaczynamy zwykle od prostych przesłanek i dopisujemy kolejne linie aż do uzyskania celu. W dekompozycji celu zaczynamy natomiast od sekwentu, który chcemy otrzymać, i pytamy, jakie przesłanki mogłyby prowadzić do niego przez jedną z reguł wnioskowania.

Schematycznie:

$$\begin{array}{ccccc} \text{problem} & & \text{wybór} & & \text{podproblemy} \\ S & \longrightarrow & \text{reguły } R & \longrightarrow & S_1, \dots, S_n \end{array}$$

przy czym warunkiem poprawności rozbicia jest

$$S \in R(S_1, \dots, S_n).$$

Rozbicie z założenia powinny generować podproblemy o dowodach prostszych niż dowód problemu początkowego. Co więcej trzecia strategia nie zastępuje generowania dowodów liniowych, ale organizuje je w większą strukturę: zamiast od razu znaleźć jeden pełny dowód sekwentu, system może rozbić go na mniejsze sekwenty i dla każdego z nich uruchomić szukanie dowodu w formie liniowej. Szukanie to odbywa się przy pomocy strategii drugiej.

3.4.3. Format zapytania o rozbicie

W trzeciej strategii model językowy jest używany nie tylko do generowania kolejnych linii dowodu, lecz także do proponowania rozbić celu na podproblemy. Jest to osobne zadanie generatywne. W przypadku dowodzenia liniowego model otrzymuje zapis celu oraz dotychczasowy fragment dowodu i ma zaproponować jego dalszy ciąg. W przypadku rozbijania problemu model otrzymuje sekwent, który chcemy udowodnić, i ma wskazać regułę wnioskowania oraz listę podproblemów, których udowodnienie pozwoli otrzymać sekwent wyjściowy.

Z tego powodu w systemie wykorzystywane są dwa modele językowe o tej samej architekturze transformera, ale trenowane na różnych typach danych. Pierwszy model uczy się generować dowody w notacji tekstowej, czyli dopisywać kolejne linie częściowego dowodu. Drugi model uczy się generować rozbicia problemów na podproblemy. Oba modele korzystają z podobnej idei: proponują tekstowy kandydat, który następnie jest sprawdzany przez formalny parser i mechanizm weryfikacji poprawności. Różnią się natomiast formatem danych treningowych oraz rodzajem tekstu, który mają wygenerować.

Aby drugi model mógł być trenowany i używany w taki sam sposób jak model dowodzący, rozbicia muszą zostać zapisane w jednoznacznej formie tekstowej. Format ten powinien określać trzy informacje: sekwent, który jest rozbijany, regułę wnioskowania używaną do rozbicia oraz listę sekwentów-przesłanek otrzymanych po zastosowaniu tej reguły.

W systemie rozbicie zapisywane jest jako tekst o następującym formacie:

Smash: S
 With: R
 To: $S_1,$
 $\dots$
 S_n

gdzie S jest sekwentem, który chcemy rozbić, R jest nazwą reguły wnioskowania, a $S_1, \dots, S_n$ są sekwentami-przesłankami otrzymanymi po rozbiciu.

Pierwsza linia określa problem wyjściowy. Druga linia wskazuje regułę, za pomocą której problem ma zostać rozbity. Kolejne linie zawierają listę podproblemów.

Jeżeli reguła nie wymaga żadnych przesłanek (np reguła **Assumption**), lista podproblemów jest pusta.

Przykładowy schemat rozbicia można zapisać następująco:

Smash: $\Gamma \vdash \varphi \rightarrow \psi$
With: $\rightarrow$ -**introduction**
To: $\Gamma, \varphi \vdash \psi$

Taki zapis oznacza, że problem udowodnienia implikacji został zastąpiony problemem udowodnienia jej prawej strony przy dodatkowym założeniu jej lewej strony.

3.4.4. Sprawdzanie poprawności rozbicia

Ponieważ rozbicia są generowane przez model językowy, każde wygenerowane rozbicie musi zostać sprawdzone formalnie. Rozbicie składa się z problemu wyjściowego S , reguły R oraz listy podproblemów $S_1, \dots, S_n$. Jest poprawne wtedy, gdy zastosowanie reguły R do podproblemów pozwala otrzymać problem wyjściowy.

W implementacji reguły wnioskowania mogą być stosowane tylko do sekwentów, które mają ten sam kontekst bazowy. Dlatego przed sprawdzeniem rozbicia wszystkie podproblemy sprowadzane są do postaci z pustym kontekstem bazowym. Cały ich kontekst logiczny zostaje przeniesiony do kontekstu dodatkowego. Jeżeli podproblem ma kontekst bazowy Γ_b , kontekst dodatkowy Γ_d i tezę φ , to zastępujemy go sekwentem

$$\emptyset; \Gamma_b, \Gamma_d \vdash \varphi.$$

Średnik oddziela tutaj kontekst bazowy od kontekstu dodatkowego.

Tak samo przekształcany jest problem wyjściowy S . Nie jest to potrzebne do samego zastosowania reguły, ale upraszcza końcowe porównanie: wynik zastosowania reguły do przekształconych podproblemów porównujemy z problemem wyjściowym zapisanym w tej samej konwencji.

Procedura sprawdzania wygląda więc następująco:

SPRAWDŹROZBICIE($S, R, S_1, \dots, S_n$) :

sprowadź S do postaci z pustym kontekstem bazowym,
 sprowadź każde S_i do postaci z pustym kontekstem bazowym,
 sprawdź pomocniczo, czy każde S_i jest sekwentem tautologicznym,
 wyznacz dodatkowe argumenty potrzebne regule R ,
 spróbuj zastosować R do przekształconych podproblemów,
 jeżeli zastosowanie reguły zgłasza błąd, odrzuć rozbicie,
 jeżeli otrzymany sekwent jest różny od przekształconego S , odrzuć rozbicie,
 w przeciwnym przypadku zaakceptuj rozbicie.

Dodatkowe argumenty są potrzebne dla tych reguł, których zastosowanie nie jest jednoznacznie wyznaczone przez same przesłanki. Na przykład przy regule $\forall I_1$

trzeba wskazać prawy składnik alternatywy, a przy *RAA* formułę, której dowodzimy przez sprowadzenie jej negacji do sprzeczności. W takich przypadkach brakujące dane są odczytywane z problemu wyjściowego.

Sprawdzanie tautologiczności podproblemów jest elementem pomocniczym. Nie jest konieczne do stwierdzenia, czy rozbiecie pasuje do schematu reguły, ale pozwala odrzucić rozbiecia prowadzące do podproblemów, których system i tak nie byłby w stanie udowodnić. Jest to uproszczenie przyjęte ze względu na prototypowy i eksperymentalny charakter systemu oraz niewielki rozmiar rozważanych formuł. Nie zastępuje ono generowania dowodu: zaakceptowany podproblem nadal musi mieć formalny dowód, którego kolejne linie są sprawdzane przez parser i reguły wnioskowania.

3.4.5. Generowanie rozbić przez model

Rozbiecia są generowane przez model językowy w podobny sposób jak kolejne linie dowodu. Model otrzymuje początek tekstu zawierający linię **Smash:** z opisem sekwentu, który ma zostać rozbity, a następnie próbuje dopisać dalszą część opisu: wybraną regułę oraz listę podproblemów. Wygenerowany kandydat jest następnie sprawdzany przez procedurę weryfikującą rozbiecie.

Schemat generowania pojedynczego poprawnego rozbiecia wygląda następująco:

WYGENERUJROZBICIE(*S*) :

utwórz prompt zaczynający się od **Smash:** *S*,
 wygeneruj dalsze linie opisu rozbiecia,
 sprawdź, czy tekst ma pełny format rozbiecia,
 sprawdź formalną poprawność rozbiecia,
 jeżeli rozbiecie jest poprawne, zwróć je,
 w przeciwnym przypadku ponów próbę generacji.

W praktyce dla jednego problemu można wygenerować kilka rozbić. Część z nich może się powtarzać, ponieważ model często wskazuje te same najbardziej naturalne reguły. Powtórzenia nie są całkowicie bezużyteczne: można je traktować jako informację o tym, które rozbiecia model wybiera najchętniej. Dlatego częstości występowania poszczególnych rozbić mogą zostać użyte jako wagi przy późniejszym wyborze gałęzi drzewa poszukiwania.

3.4.6. Dwa typy wierzchołków drzewa

Aby połączyć dekompozycję celu z generowaniem dowodów częściowych, system buduje drzewo poszukiwania. W drzewie tym występują dwa rodzaje wierzchołków.

Pierwszy rodzaj wierzchołków reprezentuje problemy dowodowe, czyli sekwenty do udowodnienia. Będziemy je nazywać wierzchołkami typu LP , ponieważ odpowiadają obiektom reprezentującym pojedynczy sekwent. Dziećmi takiego wierzchołka są możliwe rozbicia danego sekwentu.

Drugi rodzaj wierzchołków reprezentuje konkretne rozbicia problemu na podproblemy. Będziemy je nazywać wierzchołkami typu SP . Dziećmi wierzchołka typu SP są sekwenty-przesłanki występujące w danym rozbiciu.

Struktura drzewa jest więc naprzemienna:

$$\begin{array}{ccccc} \text{sekwent} & & \text{rozbicie} & & \text{podsekwent} \\ LP & \longrightarrow & SP & \longrightarrow & LP_1, \dots, LP_n \end{array}$$

Wierzchołek typu LP jest rozwiązany, jeżeli udało się bezpośrednio wygenerować dowód odpowiadającego mu sekwentu albo jeżeli przynajmniej jedno z jego rozbić prowadzi do rozwiązanych podproblemów. Wierzchołek typu SP jest rozwiązany, jeżeli rozwiązane są wszystkie podproblemy występujące w danym rozbiciu.

3.4.7. Wartość wierzchołka

Każdemu wierzchołkowi drzewa przypisywana jest wartość logiczna informująca, czy problem zakodowany w tym wierzchołku został już rozwiązany. Wartość ta jest obliczana inaczej dla obu typów wierzchołków.

Dla wierzchołka typu LP , który reprezentuje pojedynczy sekwent, wystarczy znaleźć jeden sposób jego udowodnienia. Może to być dowód wygenerowany bezpośrednio albo jedno z poprawnych rozbić, którego wszystkie podproblemy zostały rozwiązane. Dlatego wartość wewnętrznego wierzchołka typu LP jest alternatywą wartości jego dzieci:

$$Value(LP) = \bigvee_{C \in Children(LP)} Value(C).$$

Dla wierzchołka typu SP , który reprezentuje konkretne rozbicie, sytuacja jest inna. Rozbicie jest użyteczne dopiero wtedy, gdy uda się rozwiązać wszystkie jego podproblemy. Dlatego wartość wewnętrznego wierzchołka typu SP jest koniunkcją wartości jego dzieci:

$$Value(SP) = \bigwedge_{C \in Children(SP)} Value(C).$$

Takie wartościowanie odpowiada naturalnej interpretacji drzewa poszukiwania. Wierzchołek problemu wymaga znalezienia przynajmniej jednej skutecznej metody dowodu. Wierzchołek rozbicia wymaga natomiast rozwiązania wszystkich problemów, na które dany problem został rozłożony.

3.4.8. Rozwijanie liścia

Liść typu LP reprezentuje sekwent, dla którego system nie zna jeszcze ani bezpośredniego dowodu, ani żadnego rozbicia na podproblemy. Rozwinięcie takiego liścia polega na podjęciu dwóch kolejnych prób.

Najpierw system próbuje znaleźć bezpośredni dowód sekwentu za pomocą wcześniejszej strategii przeszukiwania wielu częściowych dowodów. Jeżeli taka próba zakończy się sukcesem, wierzchołek zostaje oznaczony jako rozwiązany. Nie ma wtedy potrzeby rozbijania go na podproblemy.

Jeżeli bezpośredniego dowodu nie uda się znaleźć w przyjętych limitach, system próbuje wygenerować jedno lub kilka rozbić tego sekwentu. Poprawne rozbicia są dodawane jako dzieci rozwijanego wierzchołka, a do każdego z nich dodawane są następnie dzieci odpowiadające podproblemom danego rozbicia.

Schemat rozwijania liścia można zapisać tak:

RozwińLiść(LP) :

- spróbuj znaleźć bezpośredni dowód sekwentu LP ,
- jeżeli dowód został znaleziony, oznacz LP jako rozwiązany,
- w przeciwnym przypadku wygeneruj poprawne rozbicia LP ,
- dodaj każde rozbicie jako dziecko typu SP ,
- dla każdego rozbicia dodaj jego podproblemy jako dzieci typu LP ,
- zaktualizuj wartości logiczne odpowiednich wierzchołków.

W ten sposób system łączy dwa tryby działania. Dopóki możliwe jest znalezienie dowodu bezpośrednio, korzysta z generowania liniowego. Dopiero gdy to się nie uda, przechodzi do bardziej strukturalnego kroku, czyli dekompozycji celu.

3.4.9. Rekurencyjne przeszukiwanie drzewa

Ostateczna strategia polega na rekurencyjnym przeszukiwaniu opisanego drzewa. Jeżeli aktualny wierzchołek jest już oznaczony jako rozwiązany, procedura kończy się sukcesem. Jeżeli jest nierozwiązanym liściem typu LP , system próbuje go rozwinać. Jeżeli jest wewnętrznym wierzchołkiem typu LP , wystarczy znaleźć rozwiązanie dla jednego z jego dzieci. Jeżeli natomiast jest wierzchołkiem typu SP , trzeba znaleźć rozwiązania dla wszystkich jego dzieci.

Pseudokod można zapisać następująco:

```

PRZESZUKAJ( $v$ ) :
  jeżeli  $Value(v) = True$  :
    zwróć  $True$ ,
  jeżeli  $v$  jest liściem typu  $LP$  :
    ROZWIŃLIŚĆ( $v$ ),
    zwróć  $Value(v)$ ,
  jeżeli  $v$  jest wewnętrznym wierzchołkiem typu  $LP$  :
    wybierz jedno z jego dzieci typu  $SP$ ,
    jeżeli PRZESZUKAJ( $SP$ ) =  $True$  :
      zaktualizuj wartość  $v$ ,
      zwróć  $Value(v)$ ,
  jeżeli  $v$  jest wierzchołkiem typu  $SP$  :
    dla każdego dziecka  $u$  wierzchołka  $v$  :
      jeżeli PRZESZUKAJ( $u$ ) =  $False$  :
        zwróć  $False$ ,
    zaktualizuj wartość  $v$ ,
    zwróć  $True$ .

```

Ta procedura odzwierciedla różnicę między dwoma typami wierzchołków. W przypadku wierzchołka typu LP wystarczy znaleźć jedną skuteczną drogę dowodu, dlatego wybór między rozbiciami ma charakter alternatywny. W przypadku wierzchołka typu SP dane rozbicie narzuca listę podproblemów, z których każdy musi zostać udowodniony, dlatego przeszukiwanie ma charakter koniunkcyjny.

Strategia trzecia jest więc połączeniem wcześniejszych metod z planowaniem struktury dowodu. Model językowy jest używany zarówno do generowania dowodów sekwentów, jak i do proponowania rozbić celów na podproblemy, natomiast formalny weryfikator sprawdza poprawność wszystkich wygenerowanych kroków. Dzięki temu system może wykorzystywać elastyczność modelu językowego, ale nie rezygnuje z formalnej kontroli poprawności.

3.4.10. Sklejanie dowodów

Opisane wcześniej drzewo poszukiwania pozwala stwierdzić, czy dany problem został rozwiązany, ale sama wartość logiczna wierzchołka nie jest jeszcze końcowym dowodem w notacji tekstowej. Jeżeli problem został rozwiązany przez bezpośrednie wygenerowanie dowodu, sytuacja jest prosta: wystarczy zwrócić znaleziony dowód. Jeżeli jednak problem został rozwiązany przez rozbicie na podproblemy, trzeba jeszcze połączyć dowody tych podproblemów w jeden dowód problemu wyjściowego.

Operacja ta odpowiada odwróceniu procesu rozbijania. Wierzchołek typu SP przechowuje regułę wnioskowania oraz listę podproblemów. Jeżeli dla każdego z tych

podproblemów znaleziono dowód, to można najpierw umieścić te dowody jeden po drugim, a następnie dopisać na końcu linię, która stosuje regułę zapisaną w rozbiciu do ostatnich linii odpowiednich poddowodów. W ten sposób z dowodów przesłanek otrzymujemy dowód konkluzji.

Najprostszy przypadek zachodzi wtedy, gdy reguła użyta w rozbiciu nie wymaga dodatkowych założeń lokalnych. Wtedy procedura sklejania wykonuje trzy kroki:

SKLEJDOWODY($D_1, \dots, D_n, R, S$) :

- przenumeruj linie dowodów $D_1, \dots, D_n$,
- połącz je w jeden ciąg linii,
- dopisz linię stosującą regułę R do końcowych linii poddowodów,
- sprawdź parserem poprawność otrzymanego dowodu,
- zwróć połączony dowód.

Przenumerowanie linii jest konieczne, ponieważ każdy poddowód jest generowany jako osobny fragment zaczynający się od linii numer 1. Po połączeniu kilku takich fragmentów odwołania do wcześniejszych linii muszą zostać przesunięte o liczbę linii występujących w poprzednich poddowodach. Bez tego dwa różne poddowody mogłyby zawierać linie o tych samych numerach, a odwołania w regułach wskazywałyby nieprawidłowe miejsca.

Dodatkowym problemem jest obecność kontekstu bazowego w dowodach podproblemów. Poddowód sekwentu z założeniami może być zapisany w formie, w której założenia te występują w nagłówku promptu jako kontekst bazowy. Przy sklejaniu kilku poddowodów wygodniej jest jednak mieć jeden zwykły ciąg linii dowodu, bez osobnych nagłówków dla każdego fragmentu. Dlatego przed połączeniem poddowodów kontekst bazowy zostaje usunięty z nagłówka i wprowadzony jawnie jako początkowe linie dowodu. Założenia z kontekstu są więc zamieniane na zwykłe linie, do których można się później odwoływać.

Schematycznie można to przedstawić następująco:

dowody usunięcie przenumerowanie dopisana pełny
podproblemów $\longrightarrow$ kontekstów bazowych $\longrightarrow$ linii $\longrightarrow$ reguła sklejająca $\longrightarrow$ dowód .

Niektóre reguły wymagają przy sklejaniu dodatkowej ostrożności. Dotyczy to przede wszystkim reguł, które usuwają pewne założenie z kontekstu, takich jak wprowadzenie implikacji, wprowadzenie negacji oraz *RAA*. W takich przypadkach sama końcowa linia poddowodu nie wystarcza, ponieważ reguła musi odwołać się również do linii wprowadzającej odpowiednie założenie lokalne. Dlatego procedura sklejania wprowadza najpierw dodatkową linię odpowiadającą temu założeniu, a dopiero potem dopisuje linię stosującą właściwą regułę.

Przykładowo, przy sklejaniu dowodu dla implikacji $\varphi \rightarrow \psi$ poddowód prowadzi do ψ w kontekście rozszerzonym o φ . Aby otrzymać końcową linię z $\varphi \rightarrow \psi$, system musi wskazać zarówno linię, w której założono φ , jak i linię, w której uzyskano ψ .

Analogiczna sytuacja zachodzi przy wprowadzaniu negacji, gdzie lokalne założenie φ prowadzi do sprzeczności, oraz przy *RAA*, gdzie lokalne założenie $\neg\varphi$ prowadzi do sprzeczności.

Ostateczna procedura odtwarzania dowodu z drzewa działa rekurencyjnie. Jeżeli wierzchołek typu *LP* jest rozwiązaniem liściem, zwracany jest jego bezpośredni dowód po sprowadzeniu kontekstu bazowego do zwykłych linii dowodu. Jeżeli wierzchołek typu *LP* został rozwiązany przez jedno ze swoich dzieci, procedura przechodzi do tego dziecka. Jeżeli natomiast wierzchołek jest typu *SP*, najpierw rekurencyjnie odtwarzane są dowody wszystkich jego dzieci, a następnie są one sklejane przy użyciu reguły zapisanej w tym rozbieciu.

Można to zapisać następująco:

```

ODTWÓRZDOWÓD( $v$ ) :
  jeżeli  $Value(v) = False$  :
    zgłoś błąd,
  jeżeli  $v$  jest rozwiązaniem liściem typu LP :
    zwróć dowód zapisany w  $v$ ,
  jeżeli  $v$  jest wewnętrznym wierzchołkiem typu LP :
    wybierz dziecko  $u$  takie, że  $Value(u) = True$ ,
    zwróć ODTWÓRZDOWÓD( $u$ ),
  jeżeli  $v$  jest wierzchołkiem typu SP :
    dla każdego dziecka  $u_i$  oblicz  $D_i = ODTWÓRZDOWÓD(u_i)$ ,
    zwróć SKLEJDOWODY( $D_1, \dots, D_n, R, S$ ).

```

Po sklejeniu dowód jest ponownie przekazywany parserowi. Dzięki temu nawet jeżeli błąd pojawi się dopiero na etapie przesuwania indeksów, usuwania kontekstów bazowych albo dopisywania końcowej reguły, zostanie wykryty przed zwróceniem wyniku. Ostatecznie system zwraca więc nie tylko informację, że drzewo poszukiwania zawiera rozwiązanie, lecz także jeden konkretny dowód w tej samej notacji tekstowej, która była używana podczas trenowania modelu i sprawdzania częściowych dowodów.

3.5. Strategia trzecia: dekompozycja celu i drzewo poszukiwania

3.5.1. Motywacja

3.5.2. Rozbicie problemu na podproblemy

3.5.3. Format zapytania o rozbicie

3.5.4. Sprawdzanie poprawności rozbicia

3.5.5. Generowanie rozbić przez model

3.5.6. Dwa typy wierzchołków drzewa

3.5.7. Wartość wierzchołka

3.5.8. Rozwijanie liścia

3.5.9. Rekurencyjne przeszukiwanie drzewa

3.6. Podsumowanie strategii

Rozdział 4.

3 strategie

Celem tej pracy nie jest jedynie skonstruowanie modelu językowego generującego dowody w dedukcji naturalnej, lecz także zbudowanie systemu, który w pewnym stopniu naśladuje sposób, w jaki człowiek dochodzi do poprawnego dowodu. Sam model językowy może generować ciągi tekstu przypominające formalny dowód, jednak nie gwarantuje to ich poprawności logicznej. Z tego powodu każdy wygenerowany krok musi być na bieżąco weryfikowany przy użyciu opisanej wcześniej funkcji `parseProof`.

Drugim problemem jest to, że konstruowanie dowodu rzadko ma charakter całkowicie liniowy. W praktyce często wymaga ono licznych prób, cofania się i wybierania innych ścieżek rozumowania, zanim zostanie znaleziony zapis końcowy. Dlatego druga wersja systemu rozszerza prostą generację kolejnych linii o mechanizm przeszukiwania przestrzeni możliwych kroków dowodowych.

Wreszcie, dla trudniejszych twierdzeń samo lokalne przeszukiwanie kolejnych kroków może okazać się niewystarczające. System może trafiać na ślepe uliczki, ponieważ na początkowym etapie dowodu nie zawsze wiadomo, która strategia doprowadzi do celu. Z tego względu trzecia wersja systemu wprowadza możliwość rozbijania problemu na prostsze podproblemy, które następnie mogą zostać połączone odpowiednią regułą wnioskowania w rozwiązanie problemu wyjściowego.

System był rozwijany etapami. Najprostsza wersja zajmuje się wyłącznie generowaniem kolejnych linii dowodu i sprawdzaniem ich poprawności. Druga wersja dodaje mechanizm przeszukiwania metodą prób i błędów. Ostateczna wersja rozszerza ten schemat o dekompozycję celu na podproblemy. Każda kolejna wersja wykorzystuje poprzednią jako komponent składowy. W dalszej części rozdziału omawiam kolejno wszystkie trzy strategie.

Pierwsza wersja systemu zawiera się w funkcji `silliest_generate_proof`. Jej argumenty to `bundle` będący obiektem klasy `NanoGPTBundle` którym będziemy generować tekst dowodu, `prompt` który jest zmienną tekstową, która koduje twierdzenie, które mamy udowodnić, i ewentualnie początek dowodu, zmienna `temperature`

typu `float`, która pozwala kontrolować poziom determinizmu generacji oraz zmienna `max_new_lines` typu `int`, która określa ile linii dowodu chcemy wygenerować i zmienna `max_failed_attempts`, która oznacza ile maksymalnie kolejnych nieudanych prób dopisania linii dowodu akceptujemy. Każdy prompt zaczyna się od słów **Prove that:** a następnie podana jest formuła rachunku zdań w formie infiksowej. Jeżeli kontekst bazowy w generowanym dowodzie ma być niepusty to w drugiej linii znajdują się słowa: **assuming that:** po których wypisana jest pierwsza z formuł bazowego kontekstu. Kolejne formuły są wypisywane po przecinku i każda w nowej linii. Następnie w kolejnych liniach jest dopisywany dowód.

Dowód dopisujemy w pętli, która kończy się kiedy ostatnia linia koduje sekwent, który chcemy udowodnić, liczba nowych linii przekracza `max_new_lines` lub ilość kolejnych nieudanych prób dopisania linii dowodu przekracza `max_failed_attempts`. W każdej iteracji pętli próbujemy dodać jedną linię dowodu. Linia jest odrzucana jeżeli występują w niej zmienne inne niż w docelowym sekwencie, linia nie spełnia warunków składniowych lub jeśli parsowanie dowodu z nową linią zwraca błąd. Jeżeli linia nie jest odrzucana jest dopisywana do końca dowodu i zerowany jest licznik odrzucanych linii. Funkcja zwraca `prompt_str` z dopisanymi liniami dowodu (linie te nie muszą tworzyć pełnego dowodu tezy, ale intuicyjnym celem tej funkcji jest po prostu zrobienie pewnego postępu w dowodzie).

Druga wersja systemu jest zaimplementowana w funkcji `find_proof_little_smarter`. Idea tej funkcji sprowadza się do tego, że zamiast tworzyć jeden dowód, tworzymy listę potencjalnych prób dowodu (na początku jest w niej jedynie prompt z zakodowanym problemem do udowodnienia) i w każdej iteracji pętli głównej programu losujemy któryś element tej listy (prawdopodobieństwa są tak ustawione, by im wyższe jest średnie prawdopodobieństwo tokenów w danej próbie dowodu tym wyższe jest prawdopodobieństwo wylosowania danej próby) i przy pomocy funkcji `silliest_generate_proof` dopisujemy do tej próby kolejną linię dowodu i jeżeli próby dowodu po dopisaniu tej linii nie ma w liście prób dowodów dopisujemy ją do niej. Przejdźmy teraz do bardziej szczegółowego omówienia implementacji omawianej funkcji.

`find_proof_little_smarter` jako argumenty przyjmuje obiekt `textttbundle` typu `NanoGPTBundle`, który odpowiada za generowanie kolejnych tokenów, zmienną tekstową `prompt` która koduje sekwent do udowodnienia w notacji takiej samej jak w `silliest_generate_proof`, zmienną `temperature` typu `float` która służy do regulacji poziomu determinizmu i losowości podczas generowania kolejnych tokenów, oraz zmienną `max_iter` typu `int`, która określa ile maksymalnie iteracji głównej pętli dopuszczamy.

Na początku funkcja parsuje `prompt` do obiektu `goal` klasy `LittleProblem`. Bazowy kontekst `goal` jest przechowywany w liście `base_context`. Następnie tworzymy listę `prompts` w którym umieszczamy oryginalny prompt oraz tabelę `probabilities_wild`, która tworzy listę liczb potrzebnych do obliczenia, prawdopodobieństwa losowania

każdego elementu `prompts`, które trzymamy w tabeli `probabilities`. W pętli głównej losujemy element `prompts` i przy pomocy `silliest_generate_proof` generujemy kolejną linię wylosowanej próby dowodu. Tak zaktualizowany prompt zapisujemy w zmiennej `new_prompt_str`. Dopisujemy ją do wylosowanego prompta, a następnie przy pomocy funkcji `extract_proof_part` wyciągamy z tak powstałego prompta do zmiennej `new_proof` część kodującą dowód. Sprawdzamy czy numeracja wierszy w `new_proof` jest poprawna (jeżeli nie jest wylapujemy wyjątek i przechodzimy do kolejnej iteracji), a następnie parsujemy dowód do zmiennej `parsedProof` przy pomocy funkcji `parseProof` i używając kontekstu `base_context` jako kontekstu bazowego. Jeżeli podczas parsowania otrzymamy błąd wylapujemy wyjątek i przechodzimy do następnej iteracji pętli. W przeciwnym przypadku jeżeli ostatnia sparsowana linia `parsedProof` jest taka sama jak sekwent docelowy to zwracamy `new_prompt_str`. Jeśli to jeżeli `new_prompt_str` nie ma w tabeli `prompts` to dopisujemy ją do niej, a następnie aktualizujemy prawdopodobieństwa w sposób opisany poniżej.

Do aktualizacji prawdopodobieństw używamy funkcji `prompt_probability_normalised`. Jako argumenty przyjmuje `bundle` które odpowiada za liczenie prawdopodobieństw kolejnych tokenów, `prompt` którego prawdopodobieństwo analizujemy. Funkcja ta z założenia służy do badania prawdopodobieństwa promptów, które składają się z zakodowania sekwentu, który mamy udowodnić, a także niepustej próby dowodu. Najpierw dzieli `prompt` na część kodującą sekwent oraz część próby dowodu, którą zapisujemy do zmiennej `proof`. Następnie iterując się po kolejnych znakach w `proof` sumujemy do zmiennej `ans_unnormalised` logarytmy prawdopodobieństwo warunkowe jego wystąpienia o ile znaki poprzedzające go w prompcie pozostają bez zmian. Prawdopodobieństwo to obliczamy przy pomocy funkcji `last_token_probability_from_bundle`. Następnie zwracamy `ans_unnormalised` podzielone przez długość części `proof`.

Sama aktualizacja prawdopodobieństw zaczyna się od aktualizacji tabeli `probabilities_wild`, poprzez dopisanie na jej końcu wartości `prompt_probability_normalised` dla używanego `bundle` oraz `new_prompt_str`. (Pole `probabilities_wild[i]` odpowiada polu `prompts[i]`). Dla promptów, które nie posiadają części odpowiadającej za próbę dowodu funkcja `prompt_probability_normalised` nie działa poprawnie toteż nie może zostać użyta do obliczenia wartości `probabilities_wild[0]` odpowiadającego prawdopodobieństwu promptu zawierającego sam problem bez żadnej linii dowodu toteż na wartość tego pola jest ustawiana średnia wartości pozostałych pól tablicy `probabilities_wild`. Ponieważ wartości `probabilities_wild` nie sumują się do 1 i nie są dodatnie do tabeli `probabilities` zapisujemy wartość funkcji `soft_max` obliczonej dla tabeli `probabilities_wild` traktowanej jako matematyczny wektor. Tak zaktualizowanej tabeli `probabilities` używamy do losowania prompta do rozwinięcia w następnej iteracji pętli. Jeżeli po całym wykonaniu pętli wciąż nie znaleźliśmy dowodu funkcja `find_proof_little_smarter` zwraca parę list `prompts` i `probabilities`.

Ostatnia strategia, w której rozbijamy problem na podproblemy zostanie omó-

wiona w następnych rozdziałach.

Rozdział 5.

Rozbijanie problemów na podproblemy

5.1. Obserwacja

W logice klasycznej rachunku zdań:

$$((\Gamma_1 \vdash \varphi_1 \quad \wedge \quad \Gamma_2 \vdash \varphi_2 \quad \wedge \quad \dots \quad \wedge \quad \Gamma_n \vdash \varphi_n) \rightarrow \Gamma \vdash \varphi)$$

$$\leftrightarrow$$

$$((\Gamma_1, \Gamma \vdash \varphi_1 \quad \wedge \quad \Gamma_2, \Gamma \vdash \varphi_2 \quad \wedge \quad \dots \quad \wedge \quad \Gamma_n, \Gamma \vdash \varphi_n) \rightarrow \Gamma \vdash \varphi)$$

Dowód

($\rightarrow$)

Jeżeli $\Gamma_i \vdash \varphi_i$, to $\Gamma_i, \Gamma \vdash \varphi_i$ ponieważ jesteśmy w stanie skonstruować dowód φ_i w kontekście Γ_i, Γ używając jedynie formuł z kontekstu Γ_i . Tak więc

$$(\Gamma_1, \Gamma \vdash \varphi_1 \quad \wedge \quad \Gamma_2, \Gamma \vdash \varphi_2 \quad \wedge \quad \dots \quad \wedge \quad \Gamma_n, \Gamma \vdash \varphi_n)$$

$$\rightarrow$$

$$(\Gamma_1 \vdash \varphi_1 \quad \wedge \quad \Gamma_2 \vdash \varphi_2 \quad \wedge \quad \dots \quad \wedge \quad \Gamma_n \vdash \varphi_n)$$

$$\rightarrow$$

$$\Gamma \vdash \varphi$$

($\leftarrow$)

Założmy nie wprost, że teza nie zachodzi. Wtedy istnieją takie konteksty $\Gamma_1, \Gamma_2, \dots, \Gamma_n, \Gamma$ i formuły $\phi_1, \phi_2, \dots, \phi_n$, że $(\Gamma_1, \Gamma \vdash \varphi_1 \wedge \Gamma_2, \Gamma \vdash \varphi_2 \wedge \dots \wedge \Gamma_n, \Gamma \vdash \varphi_n) \rightarrow \Gamma \vdash \phi$ jest prawdziwe oraz, że $(\Gamma_1 \vdash \varphi_1 \wedge \Gamma_2 \vdash \varphi_2 \wedge \dots \wedge \Gamma_n \vdash \varphi_n) \rightarrow \Gamma \vdash \phi$ jest fałszywe. Niech

$$F(\Gamma_i) := \text{koninkcja formuł w kontekście } \Gamma_i$$

Z faktu, że w logice klasycznej dla rachunku zdań każde zdanie prawdziwe dla każdego wartościowania jest dowodliwe, to $\Gamma_i \vdash \phi_i$ jest równoważne, z tautologicznością $F(\Gamma_i) \rightarrow \phi_i$.

Fałszywość $(\Gamma_1 \vdash \varphi_1 \wedge \Gamma_2 \vdash \varphi_2 \wedge \dots \wedge \Gamma_n \vdash \varphi_n) \rightarrow \Gamma \vdash \phi$ oznacza, że istnieje wartościowanie takie, że dla każdego $i \in \{1, 2, \dots, n\}$ zachodzi $F(\Gamma_i) \rightarrow \phi_i$, ale nie zachodzi $F(\Gamma) \rightarrow \phi$, ale to oznacza, że $F(\Gamma)$ dla tego wartościowania zachodzi, z kolei ϕ nie zachodzi. Ale ponieważ dla każdego $i \in \{1, 2, \dots, n\}$ zachodzi $F(\Gamma_i) \rightarrow \phi_i$ i $F(\Gamma)$ zachodzi to znaczy, że $i \in \{1, 2, \dots, n\}$ zachodzi $(F(\Gamma_i) \wedge F(\Gamma)) \rightarrow \phi_i$ i $F(\Gamma)$, czyli zachodzi

$$(\Gamma_1, \Gamma \vdash \varphi_1 \quad \wedge \quad \Gamma_2, \Gamma \vdash \varphi_2 \quad \wedge \quad \dots \quad \wedge \quad \Gamma_n, \Gamma \vdash \varphi_n) \wedge F(\Gamma)$$

, czyli ϕ zachodzi dla tego wartościowania co daje sprzeczność.

Mimo, że dowody w dedukcji naturalnej w notacji Fitcha piszemy zaczynając od przesłanek, a następnie łączymy je odpowiednimi regułami tak by otrzymać jako konkluzję rządąną tezę, to dowody w notacji drzewiastej zwykle generuje się w przeciwnym kierunku. Najpierw w korzeniu drzewa dowodu zapisujemy tezę, którą chcemy udowodnić, a następnie wybieramy regułę i przesłanki do których jeśli zastosujemy wybraną regułę możemy otrzymać szukaną tezę. Następnie próbujemy rekurencyjnie udowodnić owe przesłanki używając ich jako korzenie nowych poddrzew drzewa dowodu. Taka metoda jest szczególnie korzystna gdy formuła jest skomplikowana, i trudno przewidzieć jak zacząć dowód w notacji Fitcha tak by dało się go doprowadzić do końca.

Podobnie jak dla dowodów w notacji Fitcha rozbić problemów będziemy kodować na dwa sposoby. Z jednej strony ich logiczną strukturę będziemy przechowywać w odpowiedniej klasie, przy pomocy której będziemy kontrolować poprawność rozbić z drugiej będziemy kodować je w odpowiedniej formie tekstowej, tak by do dokonywania rozbić można było wytrenować model językowy typu **nanoGPT** notacji

Klasą która będzie przechowywać logiczną strukturę takiego rozbić będzie klasa **smashed_problem**, której pola to **problem**, które zawiera obiekt klasy **LittleProblem**, który jest problemem, który rozbijamy, zmienną **subproblems**, która jest listą przesłanek klasy **LittleProblem**, które otrzymujemy po rozbić oraz zmienną klasy **Rule**, która jest regułą, przy pomocy której dokonujemy rozbić.

Gdy mamy obiekt typu `smashed_problem` ważne jest sprawdzić, czy zakodowane w nim rozbicie jest poprawne. To znaczy, czy faktycznie stosując zakodowaną w `Rule` regułę dowodzenia do listy przesłanek w `subproblems` faktycznie możemy otrzymać `Problem` oraz czy owe przesłanki faktycznie są sekwentami tautologicznymi. W tym celu używamy funkcji `is_smashed_correctly`. Funkcja ta jako argument przyjmuje zmienną `SP` typu `smashed_problem`. Na początku funkcja sprawdza czy wszystkie pola w `SP` mają odpowiednie typy i jeżeli tak nie jest wyrzuca ona błąd `ValueError("Bad arguments")`. Następnie dla każdego elementu `SP.subproblems` sprawdza, czy koduje on sekwent tautologiczny poprzez podstawienie każdego możliwego wartościowania zmiennych. Oczywiście wadą tego podejścia jest to, że złożoność tej operacji jest wykładnicza względem liczby różnych zmiennych występujących w danym sekwenie. Należy jednak zaznaczyć, że ze względu na ograniczoną zasobów obliczeniowych dostępnych na potrzeby pracy tworzony model i tak generuje dowody jedynie stosunkowo krótkich formuł. Dla bardziej złożonych formuł możnaby sprawdzenie wszystkich możliwości zastąpić pewnymi heurystykami np. sprawdzeniu odpowiednio dużo losowych wartościowań. Jednocześnie samo rozbicie na nietautologiczne sekwentu nie jest bardzo problematyczne, w tym sensie, że zwykle wykonujemy równolegle kilka rozbić i wystarczy, że przynajmniej jedno rozbija problem na sekwentu tautologiczne. Z kolei rozbicia na sekwentu nietautologiczne nie prowadzi w dalszej części do generacji błędnych dowodów, a jedynie sprowadzają niektóre gałęzie generacji dowodu w ślepe uliczki, co spowalnia program, ale nie wpływa na jego poprawność.

Kolejnym krokiem jest sprawdzanie czy reguła `SP.Rule` może faktycznie rozbić `SP.problem` na `SP.subproblems`. Ponieważ nie wszystkie elementy `SP.subproblems` mają ten sam `base_context` co `SP.problem`, uniemożliwia bezpośrednie zastosowanie do nich `SP.Rule.top_down` najpierw tworzymy tabelę `unnormalised_subproblems` której *i*-ty element jest obiektem klasy `LittleProblem`, którego kontekst bazowy to `SP.problem.assumptions.base_context`, z kolei kontekst dodatkowy to wszystkie formuły zawierające się w kontekście bazowym i dodatkowym obiektu `SP.subproblems[i]`, które nie znajdują się w `SP.problem.assumptions.base_context`. Konkluzja `unnormalised_subproblems` jest taka sama jak w `SP.subproblems[i]`. Następnie stosujemy `SP.Rule.top_down` której argumentami są lista obiektów klasy `LittleProblem` o tych samych kontekstach bazowych: `unnormalised_subproblems` i odpowiednio wydedukowane z `SP.problem` pozostałe argumenty. Jeśli wynik tej funkcji jest taki sam jak `SP.problem` to funkcja `is_smashed_correctly` zwraca `True`, zaś w przeciwnym razie `False`.

Format tekstowy zapisu rozbięcia formuły jest następujący: w pierwszej linii mamy tekst `Smash:` , po którym zostaje zapisany sekwent który chcemy rozbić. Druga linia zaczyna się od napisu `With:` po którym następuje nazwa reguły której używamy do rozbięcia. Trzecia linia zaczyna się od tekstu `To:` po którym w następuje pierwsza przesłanka na którą zostaje rozbity rozbijany sekwent. Kolejne sekwentu rozbięcia są zapisywane w kolejnych liniach po przecinku. Jeżeli Lista przesłanek jest pusta prompt kodujący rozbięcie kończy się po prostu na `To:` . Do spraw-

dzania czy dany tekst w tym formacie koduje poprawne rozbiecie używamy funkcji `check_smash_from_text`, która najpierw parsuje tekst kodujący rozbiecie do obiektu typu `smashed_problem`, a następnie sprawdza jego poprawność przy pomocy funkcji `is_smashed_correctly`. Jeżeli rozbiecie jest niepoprawne, lub podany w argumencie `prompt` nie jest w odpowiednim formacie `check_smash_from_text` wyrzuca błąd `TypeError` z odpowiednią wiadomością. Jeżeli zaś `prompt` koduje poprawne rozbiecie funkcja zwraca zakodowane rozbiecie w formie sparsowanej.

Do generowania poprawnych napisów kodujących rozbiecie służy funkcja `silliest_generate_smash`. Przyjmuje ona za argumenty zmienną `bundle` typu `NanoGPTBundle` odpowiadającą za generację kolejnych tokenów oraz zmienną tekstową `prompt` kodującą sekwent który chcemy rozbić w omawianym formacie tekstowym. W bloku `try ...except` funkcja ta generuje kolejne linie rozbiecia przy pomocy funkcji `next_line`, które dopisujemy do zmiennej `prompt`. Robimy to aż do momentu gdy tak zaktualizowany `prompt` nie będzie spełniać predykatu `is_ready_to_check_smash`, który zwraca prawdę wtedy i tylko wtedy gdy podana jako argument zmienna tekstowa `prompt`, po uprzednim usunięciu znaków nowej linii z jej końca ma nie mniej niż 3 linie i jej ostatnim znakiem nie jest `,`. Kiedy ten predykat będzie spełniony sprawdzamy najpierw przy pomocy poprawność rozbiecia zakodowanego w `prompt` przy pomocy `check_smash_from_text` i jeśli nie jest poprawne wyrzucamy odpowiedni błąd, z kolei jeśli jest poprawne sprawdzamy czy w przesłankach na które rozbiliśmy rozbijany sekwent nie pojawiają się zmienne inne niż w rozbijanym sekwencie. Jeżeli tak się dzieje zwracamy wyrzucamy błąd. Jeśli podczas rozbijania zostanie wyłapany błąd zostawiamy w prompcie tylko pierwszą linię i znak nowej linii, a następnie powtarzamy całą procedurę, aż do momentu kiedy nie wygenerujemy poprawnego rozbiecia.

W prototypowych wersjach programu rozważane były wyłącznie dowody formuł tautologicznych, a więc sekwentów o pustym kontekście. Ograniczenie to okazało się jednak niewystarczające, ponieważ podczas rozbijania celu na prostsze podproblemy pojawiają się sekwent o niepustych kontekstach. Aby system mógł poprawnie obsługiwać takie przypadki, konieczne było rozszerzenie go o możliwość dowodzenia sekwentów z założeniami.

Z Obserwacji 1 wynika przez prostą indukcję po drzewie dowodu, że dowód $\Gamma \vdash \varphi$ w którym kontekst każdego wierzchołka zawiera Γ , istnieje wtedy i tylko wtedy gdy istnieje również dowód sekwentu zawierający w liściach sekwent o pustym kontekście. Toteż bez straty ogólności w całym dowodzie $\Gamma \vdash \varphi$ możemy przyjąć za prawdziwe założenia z kontekstu Γ . Jest to użyteczne ponieważ w procesie generowania dowodu mamy gwarancję, że nie trafimy w ślepią uliczkę w postaci odrzuceniu niektórych założeń z dowodzonego sekwentu, do których nasz model będzie miał trudność wrócić. Jest to motywacja dlaczego `Context` jest podzielony na dwie tabele `base_context` i `additional_context`.

Rozdział 6.

Klasa `node`

Aby połączyć zarówno rozbijanie problemów na podproblemy oraz generowanie dowodów w notacji Fitcha w jeden kompleksowy system będę używał klasy `node`.

`node` koduje drzewo przeszukiwań dowodu. Drzewo to ma wierzchołki dwóch rodzajów. Pierwszy z nich w polu `Interior` zawiera obiekt klasy `LittleProblem` z kolei drugi zawiera obiekt klasy `smashed_problem`. Dla uproszczenia będę wierzchołki pierwszego rodzaju nazywać `LPNode`, z kolei wierzchołki drugiego rodzaju będę nazywać `SPNode` mimo, że takie nazewnictwo nie występuje w samym kodzie programu. Dzieci rodzaju `LPNode` są rodzaju `SPNode`, z kolei dzieci rodzaju `SPNode` są rodzaju `LPNode`. Precyzyjniej dziećmi wierzchołków zawierających w polu `Interior` obiekt klasy `LittleProblem` kodujący pewien sekwent, w polu `Interior` mają obiekty klasy `smashed_problem` kodujące poprawne rozbicia tego obiektu. Z kolei dzieci wierzchołków zawierających w polu `Interior` obiekt klasy `smashed_problem` mają w polu `Interior` obiekty klasy `LittleProblem` zawierające przesłanki tego rozbicia.

Poza zmienną `Interior` klasa `node` zawiera też pole `Parent` które wskazuje na rodzica wierzchołka, pole `Children` listę dzieci, pole `Text`, które dla `LPNode` zawiera prompta z zapytaniem o dowód sekwentu w polu `Interior` lub zapytanie wraz z pełnym dowodem oraz pole `Probability`, które dla `SPNode` określa prawdopodobieństwo, że wybierzemy do analizy ten wierzchołek schodząc w dół od jego rodzica. Ostatnim polem jest zmienna boolowska `Value`. Dla liści typu `LittleProblem` ma wartość `False` jeśli w polu `Text` zawiera ona prompta jedynie z zapytaniem o dowód sekwentu i `True` jeśli w tym polu prompt składa się z zarówno zapytania jak i dowodu. Dla wierzchołków wewnętrznych rodzaju `LPNode` wartość `Value` to alternatywa wartości `Value` dzieci tego wierzchołka. Wierzchołek rodzaju `SPNode` może być liściem tylko wtedy, gdy lista przesłanek obiektu klasy `smashed_problem` w z kodowanym w jego polu `Interior` jest pusta. Wtedy też wartość pola `Value` dla tego wierzchołka jest równa `True`. Dla wierzchołków wewnętrznych rodzaju `SPNode` pole `Value` jest równe koniunkcji wartości pól `Value` jego dzieci.

Metoda `Smash_Leaf` klasy `node` odpowiada za doczepianie do dzieci rodzaju

SPNode które zawierają zakodowane rozbicia pola *Interior* swojego rodzica, a następnie doczepienia do tych dzieci ich dzieci rodzaju *LPNode* z przesłankami rozbicia kodowanego w ich rodzicach. Funkcja ta przyjmuje jako argumenty obiekt *bundle_smashing* typu *NanoGPTBundle* odpowiadający za generowanie rozbić, oraz zmienna całkowitoliczbowa *deg* oznaczająca ile rozbić (nie koniecznie różnych) które będziemy wykonywać. Na początku tworzymy prompta zaczynającego się od *Smash:*, po którym następuje zapis sekwentu, który chcemy rozbijać i znak nowej linii. Następnie *deg* razy przy pomocy metody *silliest_generate_smash* dopisujemy do tego prompta tekstową reprezentację rozbicia. Ponieważ często model umie dość dobrze znajdować najkożystniejsze rozbicie, często one się powtarzają. Dlatego tekstowe reprezentacje wygenerowanych rozbić przechowujemy jako klucze słownika *weights*, w którym *weights[i]* jest równa liczbie ile razy uruchomiona funkcja *Smash_Leaf* wygenerowała *i*. Następnie do tabeli *Children* rozbijanego wierzchołka dodajemy obiekty klasy *node*, które w polu *Interior* mają zparsowaną przy pomocy *check_smash_from_text* do obiektu klasy *smashed_problem* wygenerowaną wcześniej tekstową reprezentację rozbicia, z kolei w polu *Probability* ma iloraz liczby ile razy dana reprezentacja tekstowa została wygenerowana i liczby *deg*. Kolejnym krokiem jest dla każdego z tych dopisanych dzieci dopisanie do tabeli *Children* ich dzieci obiekty klasy *node*, które w polu *Interior* mają obiekty typu *LittleProblem* reprezentujące przesłanki rozbić zakodowanych w ich rodzicach. Załóżmy, że któreś z dzieci wierzchołka podanego jako argument dla funkcji *Smash_Leaf* po wykonaniu wcześniej omawianych kroków nie ma dzieci. Oznacza to, że sekwent którego rozbicie jest zakodowane w tym dziecku nie potrzebuje dodatkowych kroków dowodu by go udowodnić. Ustawiamy wtedy pole *Value* tego dziecka na *True* i aktualizujemy wartości pola *Value* dla wszystkich jego przodków.

Kolejną metodą klasy *node* jest funkcja *RunLeaf*. Próbuje najpierw znaleźć dowód sekwentu, który w polu *Interior* przechowuje obiekt, który ją wywołuje, a jeśli nie jest w stanie tego zrobić dokonuje jego rozbicia przy pomocy metody *Smash_Leaf*. Funkcja ta przyjmuje dwa obiekty klasy *NanoGPTBoundle*. Pierwszym z nich jest obiekt *bundle_proving* generujący dowody sekwentów, drugim zaś z kolei jest *bundle_smashing* odpowiadający za generowanie rozbić. Kolejnym argumentem jest zmienna zmiennoprzecinkowa *temperature* regulująca poziom determinizmu przy generowaniu dowodów, zmienna całkowitoliczbowa *max_iter*, która określa maksymalną liczbę iteracji prób dopisania kolejnej linii przy próbie generowania dowodu, a także *deg*, która reguluje ile rozbić (niekoniecznie różnych) mamy wykonać, jeżeli sekwentu nie uda się udowodnić. Na początku funkcja sprawdza czy wywołujący ją obiekt jest liściem i czy jest *LPNode*. Jeżeli tak nie jest funkcja zwraca wyjątek z odpowiednim komentarzem. Następnie tworzymy prompta z zapytaniem o dowód sekwentu, który znajduje się w polu *Interior* wierzchołka w którym się znajdujemy. Zapytanie to podajemy do funkcji *find_proof_little_smarter* która przy pomocy *bundle_proving* próbuje udowodnić zadany w prompcie sekwent. Jeżeli to się uda funkcja ta zwraca obiekt klasy *str* z dowodem, jeżeli zaś nie

zwraca ona tabelę prób i pomocniczych prawdopodobieństw. Sprawdzamy więc czy `find_proof_little_smarter` zwróci zmienną typu `str` a jeśli tak to w polu `Text` wierzchołka w którym się znajdujemy zapisujemy ten dowód, a następnie ustawiamy wartość jego pola `Value` na `True` i aktualizujemy wartości pola `Value` dla jego przodków. W przeciwnym razie rozbijamy obiekt którym wywołaliśmy metodę `RunLeaf` metodą `Smash_Leaf` używając `bundle_smashing` i podaną w argumencie do `RunLeaf` zmienną `deg`. Jeżeli `RunLeaf` uda się znaleźć dowód zwracamy `True`. W przeciwnym wypadku zwracamy `False`.

Kolejną metodą klasy `node` jest funkcja `TryNode`. Funkcja ta próbuje przeprowadzić cały dowód zadanego sekwentu a także jeśli to się nie uda zaktualizować drzewo tak by było bliższe znalezienia owego dowodu. Argumenty tej funkcji mają takie same nazwy, typy oraz zastosowania jak argumenty funkcji `RunLeaf`. Jeżeli znalezienie dowodu się powiedzie funkcja zwraca `True`, a w przeciwnym wypadku zwraca `False`. Działanie tej funkcji dzielimy na trzy nietrywialne przypadki przy czym przez znalezienie dowodu dla każdego z nich rozumiemy następująco. Dla przypadku liścia rodzaju `LPNode` jest to udowodnienie sekwentu zakodowanego w polu `Interior`, dla wierzchołków wewnętrznych rodzaju `LPNode` przez znalezienie dowodu rozumiemy znalezienie dowodu dla któregoś z jego dzieci, z kolei dla wierzchołków wewnętrznych rodzaju `SPNode` przez znalezienie dowodów dla wszystkich jego dzieci. Ostatnim trywialnym przypadkiem jest gdy wierzchołek wywołujący funkcję ma w polu `Value` wartość `True`, co oznacza, że dowód dla tego wierzchołka już został znaleziony i wtedy funkcja `TryNode` zwraca `True`. Jeżeli obiekt wywołujący tą funkcję jest liściem typu `LPNode` uruchamiamy dla niego metody `RunLeaf`, z pozostałymi argumentami takimi samymi jak te podane jako argumenty dla `TryNode` i zwracamy jej wartość. Jeżeli jesteśmy w wierzchołku wewnętrznym rodzaju `LPNode` losujemy jego dziecko, które jest rodzaju `SPNode` i rekurencyjnie wykonujemy `TryNode` wywołane przez to dziecko z pozostałymi argumentami takimi samymi jak jak w wywołaniu dla jego rodzica i zwracamy wartość tego wywołania. Jeżeli zaś jesteśmy w wierzchołku `SPNode` wywołujemy funkcję `TryNode` dla kolejnych jego dzieci (pozostałe argumenty dla każdego dziecka są takie same jak dla wywołania dla rodzica), aż do momentu kiedy któreś z tych wywołań zwróci `False`. Wtedy wywołanie dla rodzica tych dzieci również zwraca `False`. Jeżeli każde wywołanie dla dzieci zwróci `True` to funkcja dla rodzica również zwraca `True`.

Rozdział 7.

Generowanie korpusu

Rozdział 8.

Trenowanie i korzystanie z
modelu (NanoGPTBundle i te
wszystkie funkcje z nim
związane !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
podkreślam
WSZYSTKIE(liczenie
prawdopodobieństwa,
dodawanie tokenu, linii,
trenowanie, dotrenowanie,
ładowanie)!!!!)